

ARISTOTLE UNIVERSITY OF THESSALONIKI
FACULTY OF ENGINEERING
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING



Accelerating top-k Selection on Heterogeneous CPU, GPU and NPU Systems

DIPLOMA THESIS

Ioannis Michalainas

ioannisam@ece.auth.gr

Supervisor: Nikolaos Pitsianis

Associate Professor, School of Electrical and Computer Engineering

Thessaloniki, September 2026

Abstract

Note on this translation

Warning! This document was translated from Greek into English automatically with the help of a large language model. The original Greek text is the authoritative version of this thesis. Some passages in this translation may read awkwardly or imprecisely as a result of the automated process.

Top-k selection identifies the k largest elements of a set. It appears across a wide range of applications and is repeated often enough that its cost is noticeable. The recent integration of neural processing units into the same die as the processor makes the question of whether offloading such computations to an accelerator is worthwhile a reasonable one.

This work implements two algorithms with fundamentally different behaviour: truncated bitonic sorting, which is data-oblivious, and map-reduce selection with local heaps, which is data-dependent. The implementations targeted three architectures: a processor with AVX-512 vector extensions, a GPU programmed in CUDA and an AMD XDNA neural processing unit. They were evaluated with a unified methodology across two measurement scopes, algorithmic and end-to-end, recording execution time, energy footprint and the volume of data moved. Their resulting performance was analysed against the system's maximum theoretical capabilities.

The results show that the problem is limited by data movement on every architecture. The ranking of the two algorithms is overturned by the ratio n/k and the choice of measurement scope completely overturns the ranking of the architectures. In energy terms, the neural processing unit exhibits the lowest power draw but a disproportionately high energy consumption, since duration dominates the product. No accelerator run is Pareto-optimal. Especially for small k , offloading is measurably worse than computing on the processor.

Keywords: top-k selection, bitonic sorting networks, heterogeneous computing, neural processing unit, MLIR-AIE, SIMD vectorisation, energy assessment, roofline model

Acknowledgements

This thesis is the largest programming project I have undertaken to date. I would like to thank my supervisor, Prof. Nikolaos Pitsianis, in whose courses I discovered my interest in high performance computing and low-level programming.

I would also like to thank my family for their unwavering support throughout my studies, especially my brother Manolis, who, as a fellow engineer, helped me with valuable advice and guidance.

Ioannis Michalainas

Contents

Abstract	2
Acknowledgements	3
1 Introduction	9
1.1 Scope of the work	9
1.2 Motivation	9
1.3 Contribution	10
1.4 Structure of the work	10
2 Background and related work	12
2.1 The top-k selection problem	12
2.2 Selection algorithms	13
2.2.1 Bitonic sorting networks	13
2.2.2 Truncated bitonic sort	14
2.2.3 Selection with heaps and map-reduce	15
2.2.4 Radix selection	16
2.3 Processing architectures	17
2.3.1 CPU vector extensions	17
2.3.2 The SIMT model and GPU memory hierarchy	17
2.3.3 The AIE dataflow architecture	18
2.4 Existing implementations and libraries	18
3 Methodology and experimental setup	20
3.1 Reference system	20
3.2 Structure of the measurement harness	21
3.3 Measured quantities	22
3.3.1 Measurement scopes: algo and end-to-end	22
3.3.2 Timing	23
3.3.3 Energy measurement	24
3.3.4 Data traffic measurement	25
3.4 Roofline model	25
3.5 Baseline libraries	27
3.6 Correctness checking	27
3.7 Parameter space	28
3.7.1 Data types	28
3.7.2 Input distributions	28
3.8 Measurement conditions and reproducibility	29

4	CPU implementation	31
4.1	Design choices	31
4.1.1	Vectorisation	31
4.1.2	Vector type abstraction	32
4.1.3	Half-precision floating-point support	32
4.1.4	Parallelisation across multiple threads	33
4.2	Bitonic sort	34
4.3	Map-reduce with local heaps	35
4.4	Memory management and data traffic	37
4.5	Reference implementation	37
4.6	Limiting factors	38
5	GPU implementation	39
5.1	Design choices	39
5.1.1	Mapping onto the grid and thread blocks	39
5.2	Bitonic sort	40
5.2.1	Organisation into tiles and shared memory	40
5.3	Map-reduce with local heaps	41
5.3.1	Parallelism limit for large k	42
5.4	Memory management and data traffic	43
5.5	Reference implementation	44
5.5.1	Radix selection in CUB	44
5.6	Limiting factors	44
6	NPU implementation	46
6.1	Design choices	46
6.1.1	Development flow with MLIR-AIE	46
6.1.2	Tile topology and object_fifo	46
6.2	Bitonic sort	47
6.2.1	Compute tile kernel	48
6.2.2	Scaling to multiple tiles	48
6.3	Map-reduce with local heaps	48
6.4	Memory management and data traffic	49
6.4.1	Pipelining and the role of the host	49
6.4.2	Data encoding and types	50
6.4.3	Final correction on the host	50
6.5	Reference implementation	51
6.6	Limiting factors	51
7	Results and evaluation	52
7.1	Correctness	52
7.2	Execution time	52
7.2.1	Dependence on input size	52
7.2.2	Dependence on the parameter k	54
7.3	Comparison across architectures	56
7.4	Roofline analysis	58
7.4.1	Memory bandwidth utilisation	60
7.5	Energy assessment	61
7.6	Effect of data type	63

7.7 Sensitivity to the input distribution	64
8 Conclusions and future work	66
8.1 Conclusions	66
8.2 Limitations	67
8.3 Future work	67
A Acronyms	69
B Reproducing the measurements	70
B.1 Software requirements	70
B.2 Compilation	71
B.3 Running the experiments	71
B.3.1 Running individual steps	72
B.3.2 Execution parameters	72
B.4 Generating the plots	73

List of Figures

2.1	Selection versus sorting	12
2.2	The full bitonic network and its truncation	15
2.3	Heap-based selection and the map-reduce scheme	16
2.4	Radix selection	17
3.1	The four levels of the measurement harness	22
3.2	The two measurement scopes per architecture	23
3.3	Reading the roofline model	26
3.4	The five input distributions	29
4.1	The fundamental compare-exchange motif	32
4.2	The half-precision path	33
4.3	Fusing consecutive layers into one pass	34
4.4	Bulk rejection in map-reduce	36
5.1	Sorted block and shared-memory tile	41
5.2	The parallelism limit of map-reduce	43
6.1	Tile topology and object_fifo queues	47
6.2	The three-stage pipelining of the offload	50
7.1	Time as a function of size, under the two measurement scopes	52
7.2	Each algorithm on the three architectures	53
7.3	Time as a function of size, by value of k	54
7.4	The three algorithms on the graphics unit for k equal to 131072.	55
7.5	Speed-up over the reference implementation	56
7.6	Placement of the runs against the measured ceilings	58
7.7	Measured read bandwidth as a function of working-set size, for the three architectures.	59
7.8	Achieved bandwidth by architecture	60
7.9	Algorithmic and end-to-end time on the graphics unit	61
7.10	Net energy and power by architecture and algorithm	62
7.11	End-to-end time and energy per measured case	62
7.12	Sensitivity of time to the input distribution	65

List of Tables

3.1	The reference system on which all measurements were run.	20
3.2	The content of the two measurement scopes per architecture.	22
3.3	The data types that are swept.	28
3.4	The input distributions that are swept.	29
3.5	The conditions to which the system is pinned before every sweep. . . .	30
6.1	Time breakdown of the offload region by stage	49
7.1	Algorithmic time for the two extremes of k	54
7.2	Algorithmic time by data type	63
B.1	The execution parameters and their defaults.	73

Chapter 1

Introduction

1.1 Scope of the work

This work examines the acceleration of top-k selection, that is, finding the k largest elements of a set of n values, on three architectures: a general-purpose processor, a graphics unit and a neural processing unit integrated into the same die as the processor.

For each architecture, two algorithmic approaches with radically different execution characteristics are developed. Truncated bitonic sort executes a fixed sequence of comparisons independent of the data, while selection with local heaps under the map-reduce scheme adapts its work to the values of the input. All six implementations are measured over the same parameter space, with a common harness and common measurement semantics, so that the differences that emerge are owed to the algorithms and the hardware, not to the way of measuring.

The central question is under what conditions offloading to an accelerator pays off, weighing both the speed of the architecture and the additional cost of the process.

1.2 Motivation

Neural processing units are now integrated on the same die as the processor of modern laptops, share its main memory and are available to any application that can make use of them. Whether it is worth offloading this computation to the accelerator has become a timely question for developing software on such hardware.

They were, however, designed for a specific workload. Their programming model is organised mainly around neural-network operators, and their architecture around the assumption that every piece of data will be used many times after it is transferred. What happens when the workload does not have this shape remains largely unexplored.

Top-k selection is a suitable problem for posing this question. It is simple to state and widespread in practice, as it appears in nearest-neighbour search, in retrieval and recommendation systems, in the sampling of language models and many other

applications. At the same time, accelerators were designed for workloads with many operations per element and extensive data reuse, whereas top-k selection has minimal operations per element and no reuse at all.

1.3 Contribution

Implementations. Six implementations were built, two algorithms on three architectures, with support for five data types. On the processor, the AVX-512 vector extensions and multiple threads are exploited. On the graphics unit, the work is organised into tiles with shared memory. The neural processing unit implementation was written entirely at the level of the MLIR-AIE framework. The implementations were verified over 2700 cases in total.

Measurement harness. Each case is measured in two explicitly distinct scopes, algorithmic and end-to-end. Energy consumption is recorded with a common methodology on all three architectures, the volume of data moved is measured directly, and the performance of each run is analysed against the roofline model derived for the same machine. This methodology is independent of the top-k problem and can be applied to any workload considered for offloading to an accelerator.

Findings. Top-k selection is a data-movement problem on every architecture examined, and no implementation is limited by computational capability. The ranking of the two algorithms depends on the ratio n/k and stays the same across all architectures. The measurement scope completely overturns the ranking of the architectures. Energy-wise, the neural processing unit draws the lowest power of all implementations, and yet ends up with a particularly high energy footprint, simply because it needs far more time.

Reproducibility. The entire experimental procedure runs from scripts included in the repository. The measurement conditions are recorded alongside the results. Details are given in appendix B.

1.4 Structure of the work

Chapter 2 defines the problem, presents the algorithm families that solve it, describes the execution models of the three architectures and concludes with the gap this work is called to fill.

The methodology follows (chapter 3). It presents what is measured, with what semantics, against which baselines and under what conditions. It is a prerequisite for reading the results of chapter 7.

Next come the implementation chapters, one per architecture (chapters 4, 5, 6), with a common internal structure so that corresponding sections can be compared directly: design choices, the two algorithms, memory management, the reference implementation, limiting factors.

Chapter 7 presents and interprets the results, namely execution time, the comparison of architectures, memory footprint and energy consumption, for different parameters, data types and input distributions.

Chapter 8 closes with the conclusions, the limits within which they hold and possible directions for continuation.

Acronyms and reproduction instructions are found in the appendices.

Chapter 2

Background and related work

2.1 The top-k selection problem

Definition (Top-k selection). Given an array A of n elements from a totally ordered set and an integer $k \leq n$, find the subset of the k largest elements of A .

Although the final order of these elements is not part of the problem's definition, every implementation in this work sorts them anyway before returning them. The final sort costs $O(k \log k)$, negligible next to the $O(n)$ of selection, and yields a single, checkable answer instead of one of the many valid arrangements of the same set.

By its nature, the problem requires less information than sorting. Sorting determines the relative order of all n elements, whereas top-k selection is satisfied with separating the set into two distinct parts: the k largest elements and the remaining $n - k$, regardless of their internal order. The lower bounds confirm this; every comparison-based algorithm requires $\Omega(n \log n)$ comparisons to sort, but only $\Theta(n)$ to select with fixed k [1]. By sorting the entire input to keep k elements, an implementation does more work than it needs to (figure 2.1).

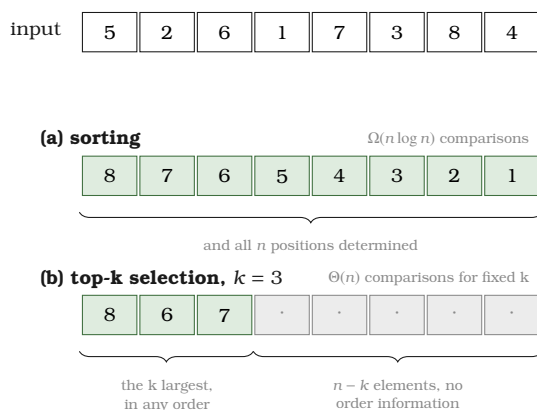


Figure 2.1: The same input under the two requirements. Sorting (a) determines all n positions, while selection (b) splits the elements into only two groups: the k largest, in any order, and the rest, for which no order information is kept.

The distance between the two problems depends on the ratio n/k . For k much smaller than n , selection is nearly linear and sorting wastes almost all of its work. As k approaches n , the two problems coincide. The entire work moves between these two extremes. Chapter 7 shows how the ranking of the algorithms is overturned as this distance is traversed.

The problem appears everywhere. The search for the k nearest neighbours requires selecting the smallest distances from a distance matrix [2]. Retrieval and recommendation systems return the k best results of a query. In language models, top- k sampling and beam search perform selection at every generation step. Although it is a small part of the overall workload each time, it is a computation repeated so often that its cost becomes noticeable.

2.2 Selection algorithms

One criterion that divides selection algorithms into families is whether the sequence of operations depends on the values of the data. An algorithm with a fixed schedule executes the same operations in the same order regardless of the input. A data-dependent algorithm adapts its course, doing less work on favourable inputs and more on adverse ones.

This distinction corresponds to the established grouping of parallel selection algorithms into sorting, partial sorting and partitioning methods [3]. Beyond theoretical complexity, it determines which hardware executes the algorithm efficiently, since parallel architectures presuppose many processing elements executing the same instruction. It also determines whether execution time is predictable.

2.2.1 Bitonic sorting networks

A bitonic sorting network is a fixed sequence of comparators. Each comparator takes two positions of the array, compares their contents and swaps them if they are in the wrong order. Which positions it compares is fixed in advance, independently of the values. It is a typical example of a data-oblivious algorithm. The same sequence of comparisons is always executed; only the outcome of each comparison changes.

The comparators are organised into layers. Within a layer, no position appears in two comparators, so all of them can execute simultaneously. The layers, however, follow one another, and their number is the depth of the network, which defines the lower bound on time on unboundedly parallel hardware.

Batcher's construction rests on the bitonic sequence [4], which first increases and then decreases, or results from a cyclic shift of such a sequence. A bitonic sequence of length $2m$ is sorted in $\log_2(2m)$ layers. Comparing each element with the one m positions away yields two bitonic sequences of length m , where every element of the first is less than or equal to every element of the second. The two halves are then sorted independently, by the same procedure, recursively.

Full sorting of n elements results from repeated application of the above idea. Pairs of sorted blocks of opposite direction form a bitonic sequence of double the length,

which is sorted to give a block of double the size. The network has $\log_2 n (\log_2 n + 1)/2$ layers and $n/2$ comparators per layer ($O(n \log^2 n)$ in total), a factor of $\log n$ more than the $O(n \log n)$ of sequential sorting algorithms. In exchange it offers fully predetermined, uniform comparisons, without any control-flow divergence. This runs equally well on vector units, on SIMT, or on dataflow arrays, which is why it remains competitive on parallel hardware despite its asymptotic disadvantage.

2.2.2 Truncated bitonic sort

Truncated bitonic sort discards elements as soon as it is proven that they cannot belong to the result [2]. The same idea, halving the active array at every step, also underlies partial-sorting implementations for graphics units [5].

Let k' be the smallest power of two greater than or equal to k . The procedure has two phases. In the first, ordinary bitonic sort is executed, but it stops as soon as sorted blocks of size k' are formed (not at the single block of size n). In the second, the blocks are merged in pairs, and of the $2k'$ elements of each pair only the k' largest are kept, while the rest are discarded permanently. This step halves the size of the active array and repeats until a single block remains, which contains the desired result. Figure 2.2 contrasts the two structures. The correctness of the discarding is immediate. If an element does not belong to the k' largest of two sorted blocks, k' elements larger than it already exist in the array, so it does not belong to the k largest of the set either.

The first phase needs $\log_2 k' (\log_2 k' + 1)/2$ layers, a count that now depends on k , instead of $\log_2 n (\log_2 n + 1)/2$. The second is applied to an array that shrinks geometrically, so its cost sums to a term linear in n . The overall complexity falls from $O(n \log^2 n)$ to $O(n \log^2 k)$.

For example, with $n = 2^{24}$ and k equal to 8, the full network has 300 layers and 2 516 582 400 comparators, while the truncated one has 90 layers and 92 274 668 comparators, that is, twenty-seven times fewer. In contrast, for $k = 131072$ it requires 279 layers and 1 441 595 392 comparators, only 1.7 times fewer than the full network. The benefit of truncation shrinks as k grows.

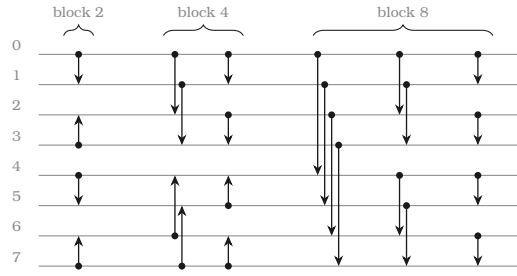
The total number of comparators in the truncated network is given by

$$\frac{n}{2} \cdot \frac{\log_2 k' (\log_2 k' + 1)}{2} + \frac{(\log_2 k' + 2)(n - k')}{2},$$

where the first term corresponds to the first phase and the second to the merge-and-truncate step.

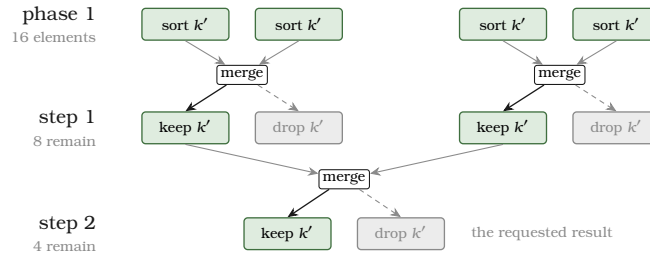
Truncation preserves data-obliviousness. The schedule of layers is computed in advance from n and k alone, so how many elements are discarded at each step is known in advance, even if which ones depends on the values.

(a) full bitonic network, $n = 8$



The arrow shows where the larger of the two elements is placed.

(b) truncated network, $n = 16, k' = 4$



Each merge takes $2k'$ elements and keeps the k' largest; the rest (dashed) are permanently discarded.

Figure 2.2: The full bitonic network and its truncation. In (a) each horizontal line is an array position and each vertical arrow a compare-exchange. The network is data-independent, since it always executes the same sequence of comparisons in the same order, regardless of the values. In (b), when only the k' best elements are requested, the first phase stops as soon as sorted blocks of size k' have formed, and in the second phase each pairwise merge keeps only the k' best of the $2k'$ compared. The array size is halved at each step, so subsequent steps cost progressively less.

2.2.3 Selection with heaps and map-reduce

Selection with heaps follows a different logic from sorting networks. It does not impose a fixed schedule; it maintains a structure with the k largest elements examined so far and updates it only when necessary.

The structure is a min-heap of capacity k . Its root holds the smallest of the k current best elements and acts as a threshold. Each new element is compared with the root in a single comparison and, if it does not exceed it, is discarded with no further work. Otherwise it replaces the root, and the heap is restored in $O(\log k)$ steps.

On random input the threshold rises quickly and the overwhelming majority of elements are discarded with one comparison, so the time approaches linear. On ascending input, however, every new element exceeds the root and forces a restoration, giving $O(n \log k)$ in the worst case.

Parallelisation follows the map-reduce scheme (figure 2.3). In the map phase the input is partitioned and each worker maintains its own local heap, without synchronising with the others, and in the reduce phase the partial results are merged into one. The scheme is attractive because the map phase requires no communication. Each local heap must first fill with k elements before it can start filtering, so the number of workers that actually contribute does not exceed the ratio n/k . On a processor

with eight threads this limit goes unnoticed, but on a graphics unit with thousands of threads it becomes the main limiting factor, as analysed in section 5.3.1.

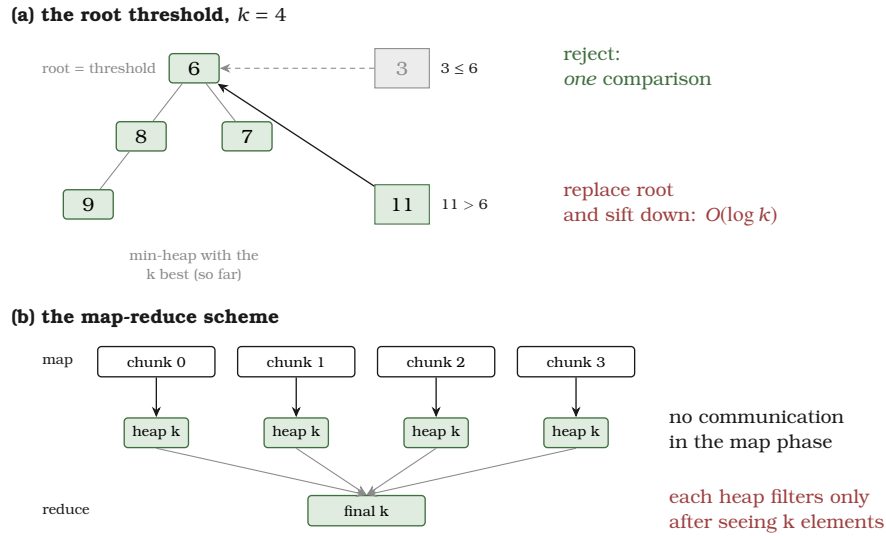


Figure 2.3: Heap-based selection and its parallelisation. In (a) the root of the min-heap acts as a threshold: an element that does not exceed it is rejected with one comparison, while one that exceeds it replaces the root and triggers a sift-down. In (b) the input is partitioned into chunks, one heap per worker, and the partial results are merged into one.

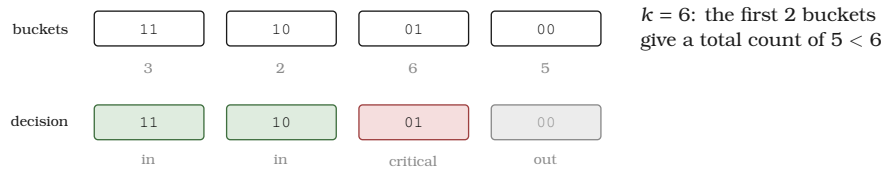
2.2.4 Radix selection

Radix selection examines the binary form of the values instead of comparing them directly. The values are distributed into buckets according to a group of digits. The bucket counts are summed, and from them it follows which bucket contains the k -th largest element. The buckets above it belong entirely to the result, those below it are discarded entirely, and the search continues only within the critical bucket, with the next group of digits (figure 2.4).

The number of passes is determined solely by the width of the data type. This is why the method has no collapse point as k grows. Each pass is, moreover, a sequential read with frequency counting, a pattern well suited to high-bandwidth architectures.

The method is also data-dependent, but in a different way from heaps: here the time depends on the distribution of digits in the values. When many elements share the same significant digits, the critical bucket narrows slowly and more passes are needed; this is the radix-adversarial distribution known in the literature, addressed with adaptive candidate-storage strategies [3]. This family was not implemented in the present work, but it is used as the basis of the graphics unit's reference library (section 2.4).

pass 1: the 2 most significant digits



pass 2



Figure 2.4: Radix selection. At each pass the elements are distributed into buckets according to a group of digits and the count of each is tallied; from the cumulative counts it follows which buckets belong entirely to the result, which are rejected, and which is the *critical* one. No comparison between values is ever performed.

2.3 Processing architectures

The three architectures expose parallelism in different ways.

2.3.1 CPU vector extensions

The modern general-purpose processor combines two forms of parallelism: multiple threads, each with its own independent instruction stream, and vector extensions, which apply the same operation to many data items simultaneously within one thread.

An AVX-512 register is 512 bits wide, that is, sixteen 32-bit integers. Beyond arithmetic operations, the repertoire includes shuffle instructions, which permute elements within the register, and min/max instructions, which select element-wise without branching. The combination of shuffle, min and max is equivalent to one sorting-network comparator. Bitonic sort is a natural candidate for vectorisation.

Data supply usually becomes the limit first, before the rate of operations has a chance to become a problem. The cache hierarchy compensates for the speed gap with main memory, but only for algorithms that reuse data while it remains at a nearby level.

2.3.2 The SIMT model and GPU memory hierarchy

The graphics unit is organised around the SIMT model. Threads are grouped into warps that execute the same instruction on different data. When the threads of a warp follow different branches, the branches execute serially with the inactive threads disabled. Control-flow divergence therefore has a direct cost, and branch-free algorithms have the advantage.

Warps are grouped into thread blocks, and blocks into a grid. Each block has shared memory, a small region managed explicitly by software, with latency far lower than that of global memory. With it, a multi-pass algorithm is turned into a single-pass

one. If the chunk of data processed by a block fits there, all intermediate steps execute without returning to global memory.

Access to global memory is efficient only when the threads of a warp request consecutive addresses, so that the individual requests are coalesced into few transactions. Finally, the graphics unit is a discrete device. Data must be transferred to and from its memory over a bus whose bandwidth is an order of magnitude lower than that of local memory.

2.3.3 The AIE dataflow architecture

The term neural processing unit has meant different things in different eras. The first proposals targeted approximate computation to save energy [6, 7]. Dedicated neural-network accelerators followed [8], whose performance-per-watt advantage was established when they were deployed at data-centre scale [9].

According to the Eyeriss design principle, moving a piece of data costs orders of magnitude more energy than the operation that will be performed on it, so design is organised around maximising local data reuse [10]. In convolution and matrix multiplication this assumption is fully vindicated, since every element loaded participates in many operations. In top-k selection each element is examined a minimal number of times, so the same assumption does not hold.

The architecture examined belongs to AMD’s AI Engine family, introduced with the Versal adaptive systems [11] and later integrated, as XDNA, into general-purpose processors [12].

The execution model is spatial, fundamentally different from the previous two. The unit consists of an array of tiles, each with its own vector processor and its own local memory. There is neither shared memory nor cache memory, so communication happens exclusively through explicit data transfer over an interconnect network.

The program is therefore expressed as a graph of compute kernels connected by streaming channels, instead of a sequence of operations on a shared address space. The programmer describes which tiles execute which kernel and how data flows between them. The abstraction this work uses for this purpose is the `object_fifo` of the MLIR-AIE framework [13], presented in section 6.1.2.

The local memory of each tile is small and fixed in size, so the computation must be split into chunks that fit there, with the partial results merged elsewhere. Moreover, feeding the array remains the responsibility of the host. It has to prepare and forward every chunk. When the computation per element is small, as in selection, the cost of feeding dominates over the cost of computation.

2.4 Existing implementations and libraries

The availability of ready-made implementations is not the same across the three architectures, and this disparity shapes both the question of this work and the choice of baselines in section 3.5.

For the processor, the C++ standard library offers `std::nth_element`, which partitions the array around the k -th element in expected linear time, and `std::partial_sort`, which additionally sorts the result by maintaining a heap of size k . Both are available everywhere and well tested, but strictly single-threaded, exploiting neither multiple threads nor vector extensions.

On the graphics unit, CUB, part of the CCCL library collection [14], provides Device-TopK, a radix selection implementation as described in section 2.2.4. It is a mature and highly optimised solution, which this work uses as a baseline, without aiming to surpass it across the entire parameter range.

How strongly the choice of algorithm depends on k is recognised in industry too. Recent work on AMD graphics units records exactly the same overturn described in section 7.2.2 and addresses it with an adaptive strategy; register-level bitonic sort for small k and radix selection for large k , with an empirically determined transition point [15].

Alongside exact methods there is also a line of approximate algorithms, which trade recall for speed and mainly target machine-learning workloads, where top- k selection proves to be a systematic bottleneck on accelerators designed for dense matrix multiplication [16]. The present work is restricted to exact selection.

For the neural processing unit no equivalent library exists: its ecosystem is oriented towards neural-network operators and does not include a top- k selection primitive. The closest available material is an official bitonic sort example on an AI Engine tile [17], which, however, performs full sorting, only for single-precision numbers, and only for sizes up to 1024 elements that fit in the local memory of one tile. The gap the present work is called to fill is exactly this: selection instead of sorting, five data types instead of one, and inputs of up to 2^{24} elements that require pipelining across multiple tiles.

Every implementation must be built from scratch at the level of MLIR-AIE [13], which makes this unit the most demanding part of the work. Moreover, there is no natural on-device baseline, so this work uses the host implementation as the reference, a choice justified in section 6.5.

The literature on top- k selection for graphics units is extensive overall, covering both sorting networks and radix-based methods [2, 15]. In contrast, the corresponding literature for AIE-type dataflow accelerators is limited, since these units are recent and their use focuses on convolution and matrix-multiplication workloads.

Chapter 3

Methodology and experimental setup

3.1 Reference system

All measurements come from one single system, which carries all three architectures. This isolates the differences between implementations from the differences between platforms. The cost is that the system is a laptop, so the three units share a common thermal and power budget.

System	Lenovo Legion 5 15AHP10
Operating system	Arch Linux x86_64, kernel 7.1.5-arch1-2
Main memory	32 GiB DDR5-5600, dual channel
CPU	AMD Ryzen 7 260
Cores / threads	8 / 16, up to 5.10 GHz
Cache	8 MiB L2, 16 MiB L3
Vector extensions	AVX-512
GPU	NVIDIA GeForce RTX 5060 Laptop
Memory	8 GiB
Software	driver 610.43.03, CUDA 13.3
Power limits	50 W default, 115 W maximum
NPU	AMD XDNA (RyzenAI-npu1)
Software	XRT 2.21.75, firmware 1.5.5.391

Table 3.1: The reference system on which all measurements were run.

The processor has eight physical cores with simultaneous multithreading. All measurements use one thread per physical core, so that no two threads share the same vector unit. The cache hierarchy sets the knee points of the measured bandwidth scale (section 3.4). Depending on the input size and the data type, the working set may fit in L2, in L3, or only in main memory, and each implementation is compared against the ceiling of its own working set.

Main memory consists of two identical 16 GiB modules in dual channel, at 5600 MT/s. A single channel would halve the bandwidth, and a memory-bound imple-

mentation would give worse results on an otherwise identical system. This was confirmed in practice after upgrading the system memory from one to two channels.

The neural processing unit has no memory of its own. It is fed from the same DDR5, through the same memory controllers as the processor, on the same silicon die, with two consequences that recur throughout this work. The cost of preparing data on the host is not separable from the cost of the computation, and the energy meter covers the whole package, not the AIE tiles individually (section 3.3.3).

The entries of table 3.1 were measured directly on the system.

3.2 Structure of the measurement harness

The harness is organised into four layers around one principle: the measurement procedure itself is common to all architectures, and only the kernel differs. The implementation itself does not affect how it is measured.

The first layer is the common C++ core, shared by all three implementations. It reads the parameters and generates the input, schedules the layers of the bitonic network and reads the energy meters. It then runs the warm-up and measurement loop, checks correctness and prints the final report. All of this happens in one place, so that the timing and energy measurements of every architecture are defined by the same code.

The second layer holds the per-architecture executables, each with the same interface for the three compared algorithms (bitonic sort, map-reduce with local heaps, reference implementation), providing only the compute function that the common measurement loop calls.

The third layer is the sweep driver. It enumerates the parameter space of section 3.7, runs each case in a separate process and records its output verbatim together with its outcome. Each case starts from clean memory, and if one fails the sweep carries on normally with the rest.

The fourth layer converts the raw results into tables and plots, embedding them directly in the final report. There is no separate output path; the values in the plots are by construction the ones the executable printed. The execution environment is stored alongside the results (section 3.8) and the commands of each stage are given in appendix B. The structure is summarised in figure 3.1.

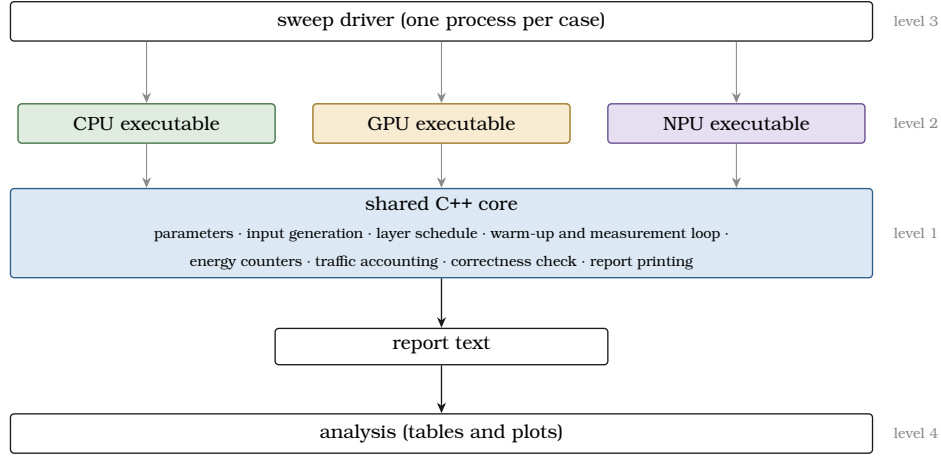


Figure 3.1: The four levels of the measurement harness. The measurement procedure is defined only once, in the shared core of the first level, and the three implementations provide exclusively the function that performs the computation.

3.3 Measured quantities

3.3.1 Measurement scopes: algo and end-to-end

An accelerator may finish the computation very quickly and still lose to a processor working in place, if it has to transfer the whole input to and from main memory. Every case is measured in two scopes.

End-to-end covers the entire call from the host’s side; everything required so that, given an input in main memory, the result ends up there too. Algo covers only the region in which the computation runs. Their difference is the cost of heterogeneity itself, as summarised in table 3.2 and in figure 3.2.

Architecture	Content of algo	Difference from end-to-end
CPU	identical to end-to-end	-
GPU	region between CUDA events	H2D/D2H transfers, memory allocations
NPU	region of the offload	device discovery, loading the offload configuration

Table 3.2: The content of the two measurement scopes per architecture.

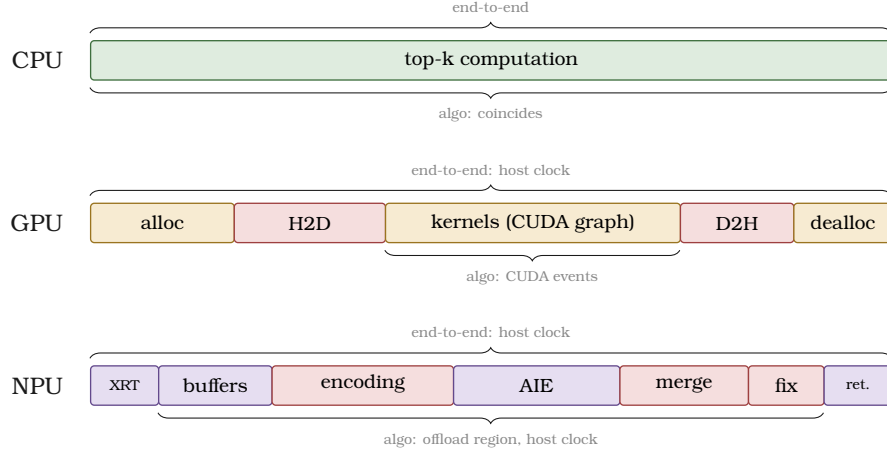


Figure 3.2: The two measurement scopes per architecture. The colour of each stage indicates which unit its work belongs to, with red for the extra cost imposed by each architecture: bus transfers for the graphics unit, encoding, merging, and final correction on the host for the neural processing unit.

On the processor the two coincide by definition, since it is the accelerator itself and there is no host cost to subtract.

The energy meter opens and closes at exactly the same points in the code as the timer. Time and energy thus refer to the same region and are directly comparable. A third, wider window around the entire measurement loop is also recorded, as an upper-bound sanity check (section 3.3.3).

End-to-end is the fair scope for comparison across architectures, since it includes the transfer cost for all of them. Algo omits the transfers, so it is only useful for comparisons within the same architecture.

3.3.2 Timing

The two scopes are not necessarily measured with the same mechanism. End-to-end is always measured with a host clock, on either side of the call, by definition the time perceived by the host. Algo is measured with whatever each architecture provides for the region of its own computation.

On the graphics unit the measurement cannot be done with a host clock. Kernels launch asynchronously and the call returns as soon as the kernel is placed in the command stream, long before it finishes. Such a clock would measure the launch, not the execution, and forced synchronisation would add its own cost. Events within the command stream, timed by the device, are used instead.

On the neural processing unit there is no such device meter. Algo is defined here by a host clock around the region of the offload. It measures allocating and preparing buffers, dispatching jobs and the final merge of the partial results. This quantity must be read as the cost of the entire offload; the computation on the AIE tiles is only one part of it.

Each case is measured over fifty repetitions, with three quantities reported per scope: the mean (the basis of the comparisons), the standard deviation (an indication of

stability) and the minimum (a lower bound). The meters have nanosecond-scale resolution, against measured regions from a few tens of microseconds to hundreds of milliseconds, so the cost of the timing itself is negligible and is not subtracted.

3.3.3 Energy measurement

The executable measures its own energy. An external tool would measure the whole process, whereas the measurement loop occupies only part of it, a part that differs systematically by architecture, introducing bias in favour of those with lower initialisation cost.

The sources differ by architecture, but the scope stays symmetric. For all of them the package energy is measured via RAPL, and for the graphics unit the card's power is added. End-to-end thus corresponds to the total cost of solving the problem for all three.

The sum, not the mean, of the fifty repetitions is reported. The RAPL counters quantise each reading, but accumulate without reset. This is why the sum stays accurate even when each individual difference is on the order of the quantisation step.

The core subfield of RAPL recorded only 6 W under full eight-thread load, while package power rose by about 53 W, a sign that it is unreliable on this processor. This is why only the package subfield is used.

Three identical runs with the NVML counter gave energy values that differed by more than a factor of four, while the time differed by only 0.2 %. The counter advances in steps of about 20 J every few tens of milliseconds, so bracketing a millisecond-scale kernel returns an arbitrary multiple of the step, which makes it useless for the graphics unit. Instead, a helper thread samples the card's instantaneous power every 2 ms and integrates it over time. The reading itself refreshes less often than the sampling period, so an algo window of a few milliseconds relies on a reading that has not had time to refresh. Summing over the 50 repetitions smooths out this inaccuracy, without eliminating it. The RAPL counters, which refresh on a millisecond scale, do not have this problem.

Beyond the gross value, net energy is also reported: gross minus what the system would consume idle over the same interval, with idle power measured before every sweep. The net figure is preferable especially for the neural processing unit, where the static consumption of the rest of the system dominates, and for the graphics unit, where it cancels out the cost of the sampling mechanism itself.

The package meter covers the entire die (host cores, NPU tiles, uncore, memory controller, all under one shared meter). A run on the neural processing unit is thus measured as the total energy of the system while it works, including data preparation on the host. The comparison of the cost of solving the problem is fair and symmetric, but it is not known how much energy the unit itself consumes for the computation, since its own consumption is never separated from that of the host. Isolating it would require external metering equipment.

3.3.4 Data traffic measurement

Every run also records the bytes it moved and the operations it performed, which are the coordinates of section 3.4. An error here does not merely give imprecise numbers; it leads to an entirely wrong conclusion about what limits the implementation.

Traffic is not estimated theoretically. It is counted directly by the code where it occurs: per group of fused passes on the processor, per kernel launch on the graphics unit, per data batch on the neural processing unit. The obvious estimate, element count times type size, assumes every element is read once, an assumption that does not hold here. Truncated bitonic sort traverses the working set repeatedly, by a factor of roughly seven times the naive estimate for k much smaller than n . The naive estimate would make every implementation appear to use only a small fraction of the available bandwidth, wrongly leading to the conclusion that none of them approaches the memory ceiling.

Wherever feasible, the count is checked by independent verification. For bitonic sort with k up to 4096, the processor and the graphics unit run the same layer schedule and compute their traffic in entirely separate code, yet arrive at the same value down to the last byte, independent evidence that the counting model is correct. For map-reduce the two values differ, as expected. Although both maintain a local heap per thread, the processor runs few threads while the graphics unit runs thousands, with a correspondingly larger working space and an additional tree merge within each thread block, so the two implementations genuinely move different volumes of data.

The accuracy of the count differs by algorithm, and every run itself states the quality of its numbers. For bitonic sort the bytes and the operations are exact, since the schedule is fully enumerable. For map-reduce both the bytes and the comparison count are lower bounds, since the merge traffic and the restoration of the local heaps depend on the data. For the reference implementation both quantities are lower bounds, because the internal passes of the libraries are not externally observable.

Operations are counted in compare-exchanges. Top- k selection performs no arithmetic operations, so FLOPs have no meaning here.

3.4 Roofline model

Execution time alone does not explain why an implementation is not faster. The roofline model [18] answers this, comparing every measurement against the two upper bounds of the machine: bandwidth β and compute capability π . Which of the two binds is a property of the relationship between machine and algorithm, not of either one separately, and is expressed by the operational intensity, the ratio of operations W to bytes Q moved:

$$I = \frac{W}{Q}, \quad P = \min(\pi, \beta I)$$

Figure 3.3 shows how it is read.

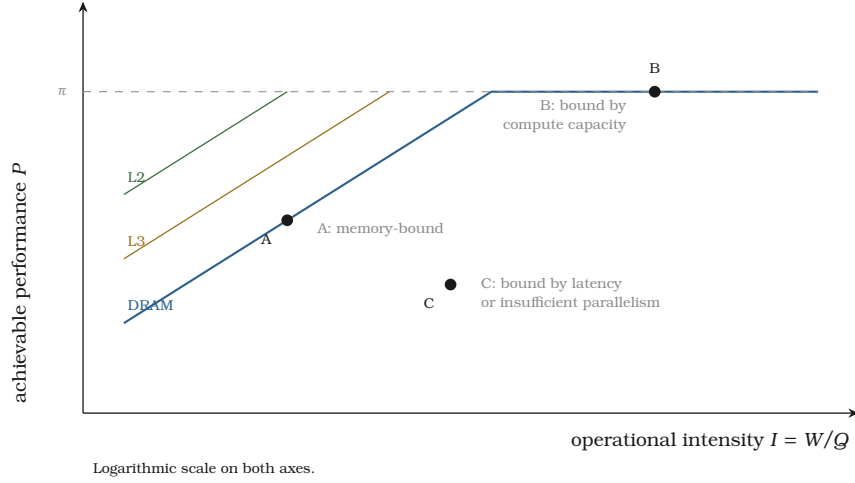


Figure 3.3: Reading the roofline model. The sloped segment corresponds to the bound βI imposed by memory bandwidth, and the horizontal one to the bound π imposed by compute capacity; the achievable performance is the minimum of the two. Because memory bandwidth depends on the size of the working set, the sloped segment is not a single line but a ladder of parallel lines, one per level of the memory hierarchy, and each measurement is compared against the one that corresponds to its own working set. Point A is memory-bound and will benefit only from a reduction in data traffic, B from compute capacity, while C is far from both ceilings: there the limiting factor is not the hardware but the organisation of the execution.

Applying the model here deliberately departs from usual practice, so that the comparisons stay honest.

Both ceilings are measured on the same machine with separate executables, so that they record what is actually achieved in practice.

The memory ceiling varies with the size of the working set, since data arrive far faster while they fit in some level of cache than from main memory. Bandwidth is therefore measured over successive sizes, starting at 32 KiB and doubling each time, and every measurement is compared against the ceiling of its own size. This approach measures bandwidth under reuse, which would seem unsuitable for an algorithm that traverses the input once. That does not hold here, though: the input is restored from a backup copy just before the timed region (section 3.8), so for working sets up to the size of the last cache level the data are already there and the measurement corresponds to a second pass. Beyond that size, the scale converges on its own to the rate of main memory.

The compute ceiling is measured in compare-exchanges, for the reason given in section 3.3.4. If every compare-exchange used the result of the previous one, a dependency chain would form, measuring latency instead of throughput, with a ceiling several times lower and every implementation wrongly appearing to touch it. The measurement kernel therefore runs 64 independent chains in parallel, alternating between three registers so that the compiler does not elide any operation.

For the neural processing unit the model means something different, since there is no AIE kernel with tunable operational intensity. The measured ceilings here

concern the feed path, not the computation on the tiles, and are read as “how fast the host can prepare the data the unit consumes”, which is the actual limitation here in any case.

3.5 Baseline libraries

The performance baseline on each architecture is an established library implementation, so that the comparison reflects the real benefit over what a programmer already has available.

For the processor, `std::partial_sort` is used. Choosing it over `std::nth_element` is not self-evident, but for k much smaller than n it is clearly faster. `std::partial_sort` wins because it maintains a heap of size k that stays in cache and rejects most elements with a single comparison, whereas `std::nth_element` traverses large parts of the array repeatedly while partitioning.

For the graphics unit, `cub::DeviceTopK` is used, which implements radix selection. It is a genuine top- k function, that is, it does not fully sort and then truncate. The comparison is therefore against an opponent that solves the same problem in an optimised way.

The AIE toolchain provides a compiler and example kernels, not an algorithms library, so no established top- k implementation exists for this hardware, neither from the manufacturer nor from third parties. The same `std::partial_sort` of the processor is used instead, run entirely on the host without any use of the unit. The question the comparison answers changes from “is our implementation better than an optimised AIE kernel” (no such kernel exists) to “is it worth moving the computation to the unit, or is what the host already has preferable”.

The shared baseline also gives an independent check of the harness. Since the same code runs on the same hardware, the two measurements ought to agree. Their agreement in the results of chapter 7 confirms that the harness introduces no systematic bias depending on the architecture.

3.6 Correctness checking

A timing measurement is meaningful only if the result is correct. Every case of every sweep is verified, and those that fail are recorded as failures instead of entering the results.

Checking is done against a separate verification implementation, which must not be confused with the libraries of section 3.5. Those measure performance; this one judges correctness. It is the simplest possible implementation: a copy of the input is fully sorted on the host and the first k elements are kept. It is slow, with no intention of competing, and only needs to be unquestionably correct.

The comparison is not the same for all data types. Integers require exact equality, while floating-point types require a tolerance, relative or absolute depending on the magnitude of the values, since different architectures perform the operations in a different order and with different intermediate precision. Every implementation returns

its result sorted, in the same order as the verification implementation (section 2.1), so the comparison is made position by position, not as a set. When both the full and the truncated network are run, the two results are also compared with each other, which independently checks that truncation does not corrupt the result.

3.7 Parameter space

Every run is fully determined by command-line parameters, none from a configuration file or environment variable. The description of a measurement is thus always complete and reproducible. The full list is in appendix B.

The input size n is a power of two, equal to 2^q , where q takes values from 3 to 24. The upper bound, about 16.8 million elements, far exceeds the cache, so the measurement concerns main memory.

The parameter k takes the values 8, 256, 4096, 131072, orders of magnitude apart, from a k so small that the heap fits in a handful of registers to a k that is a significant fraction of the input itself. Where k exceeds n , it is clamped to it.

3.7.1 Data types

Type	C++ mapping	Size	Description
int	std::int32_t	32 bit	signed integer
uint	std::uint32_t	32 bit	unsigned integer
float	float	32 bit	single-precision floating point
double	double	64 bit	double-precision floating point
half	_Float16	16 bit	half-precision floating point

Table 3.3: The data types that are swept.

The sweep covers five data types (table 3.3). The type affects the measurement in two ways: how many bytes move per element, which shifts the position of the same implementation relative to the memory ceiling, and how much a comparison costs, which is not the same for all of them.

3.7.2 Input distributions

The distribution of the input values affects execution time. For instance, heap-based algorithms do work only when a new element exceeds the root, so the order in which the values appear drastically changes their time. Five distributions are swept (table 3.4, figure 3.4). The executable also supports the normal distribution, which is not used here since trimodal covers the case of clustered values more sharply.

Distribution	Description
uniform	uniform over the value range
trimodal	three narrow clusters within the range
sorted	ascending order
reverse	descending order
adversarial	values in the lower half with ascending spikes

Table 3.4: The input distributions that are swept.

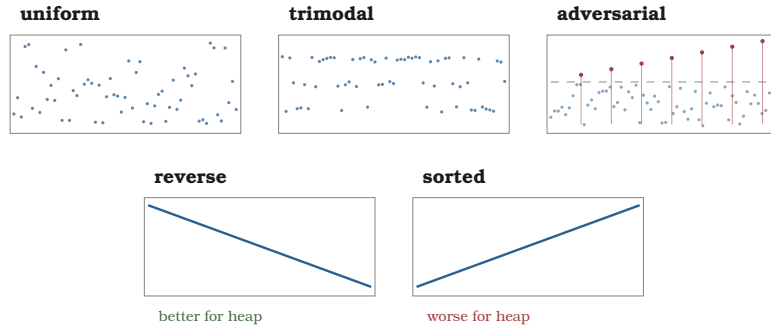


Figure 3.4: The five input distributions, with the value of each element as a function of its position in the array. The uniform and trimodal distributions have no ordering and differ only in which values occur; the two ordered ones are the extremes for heap-based selection. In adversarial the bulk of the values stays below the midpoint of the range (dashed), with isolated spikes at regular intervals.

For heap-based selection (section 2.2.3) the two ordered distributions are the two extremes. Descending is the most favourable, ascending the least favourable. Adversarial retains the unfavourable behaviour without being fully sorted. Most of the values lie in the lower half of the range, with ascending spikes at regular intervals.

Bitonic sort is data-oblivious, executing the same comparison network regardless of the values, and so serves as a witness. Any change in its time across distributions would betray an error in the measurement, since the algorithm itself has no way of causing it.

3.8 Measurement conditions and reproducibility

By default the frequency governor is powersave and frequency boost is active. The frequency varies with load and temperature, with a systematic drift over the course of the sweep as the system heats up, not random noise. The standard deviation of samples under such drifting conditions would not reflect the true uncertainty.

So before every sweep the system is pinned to a fixed, documented state (table 3.5), which is recorded before the sweep and restored afterwards.

Parameter	State during measurement
CPU frequency governor	performance
CPU frequency boost	disabled
CPU threads	8, one per physical core
GPU persistence mode	enabled
GPU power limit	fixed for the whole sweep
Cool-down period between units	15 s
Warm-up repetitions	5 per case
Measured repetitions	50 per case

Table 3.5: The conditions to which the system is pinned before every sweep.

The graphics unit’s power limit is chosen once, within the bounds of table 3.1, and kept fixed. Its value is arbitrary, but any change during the sweep would alter time and energy at the same time.

The three units share a common thermal budget (section 3.1). A 15 s cool-down period between successive units keeps each one from starting from the thermal state of the previous one. This is, however, only a partial remedy. On a laptop the thermal coupling between the units cannot be fully removed and remains an acknowledged limitation of the measurements.

Every case is run five times as warm-up and fifty times under measurement, with the mean, the standard deviation and the minimum reported. Before every repetition the input is restored from a backup copy, outside the timed region, so that the cost of the copy is not counted and every repetition starts from an identical input.

Every combination of distribution and random-number-generator seed is an independent sample, beyond the repetitions within a case. The plots aggregate these samples and depict their spread as error bands, uncertainty due to the data as distinct from uncertainty due to the measurement.

Finally, every run records the environment in which it ran (repository revision, kernel, compiler, driver and library versions of each unit). Every set of measurements thus documents the conditions of its production without depending on external records.

Chapter 4

CPU implementation

4.1 Design choices

The two implementations of this chapter share a common design basis: they vectorise the compare-exchange instead of staying with scalar code, and they go through a common abstract interface instead of being written as near-identical copies. In addition, they convert half precision to single precision for the computation, while the distribution of threads differs between the two.

4.1.1 Vectorisation

Both algorithms rely on an operation that vectorises exceptionally well. The compare-exchange of two elements is equivalent to computing the minimum and the maximum, and both exist as instructions that act on an entire vector register at once, without branches. A 512-bit register performs sixteen single-precision integer comparisons in the time of one. The instructions are referred to here by the names of the intrinsic functions of the standard [19].

The comparison itself is simple; the difficulty lies in positioning. To compare two elements that are j positions apart, they must end up in the same position of two different registers. When j is smaller than the register width, this is done by shuffling within the register. The contents are shifted by j positions, the minimum and maximum are computed, and the result is reassembled by alternately selecting from the two with a mask. The sequence “shuffle, minimum, maximum, select” is the basic motif on which the layer fusion of section 4.2 is built (figure 4.1).

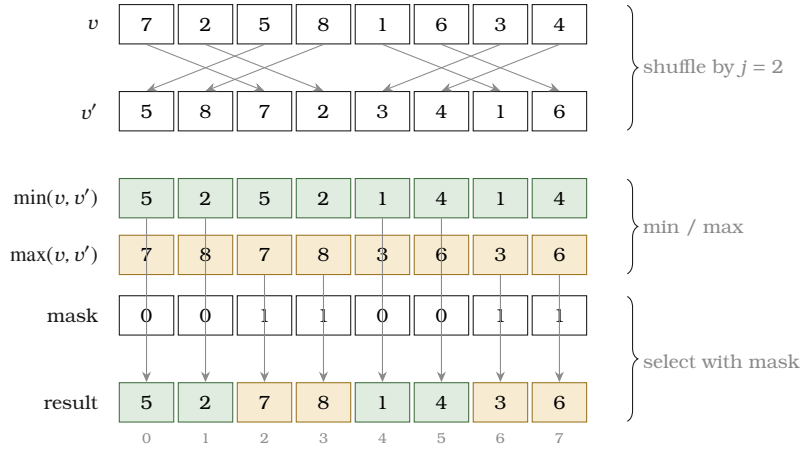


Figure 4.1: The fundamental “shuffle, min, max, select” motif for compare-exchange at distance $j = 2$, in an eight-lane register. The contents are permuted so that each element ends up opposite its counterpart, minimum and maximum are computed simultaneously for all lanes, and the result is reassembled by selecting the minimum where the corresponding mask bit is zero and the maximum where it is one. No branch intervenes and no value is examined individually.

The register width is not known at compile time. The code is compiled with all paths present. The choice is made at start-up, by querying the processor for the instructions it supports. If it supports AVX-512 it selects 512-bit registers, otherwise 256-bit ones, and otherwise, as a fallback, a scalar path. The same executable therefore runs on any machine, with the difference that on different hardware it follows a different code path and its results are not directly comparable.

4.1.2 Vector type abstraction

Vector instructions are specialised by data type. One instruction is used for the minimum of signed integers, another for unsigned, another for floating point. Five types times two register widths times two algorithms: writing the code directly would give twenty nearly identical variants, with the corresponding maintenance cost.

To avoid this, a single interface is defined. For each combination of type and width a structure is declared that always exposes the same names (vector type, width in elements, load, store, minimum, maximum, masked select, shuffle, generation of the candidate mask for map-reduce). The algorithms are written once against it, without ever referring to a specific instruction.

Adding a new type needs only a specialisation of the structure, no change to the algorithms, and a missing specialisation shows up immediately at compile time. Without a declared interface for some combination, the code automatically falls back to the scalar path. The equivalence of the paths is tested.

4.1.3 Half-precision floating-point support

The processor stores 16-bit values without having arithmetic instructions that act directly on them at the width the algorithm needs. Scalar processing would negate

any benefit. The solution exploits the fact that conversion between the two representations takes a single instruction. The half-precision specialisation of the interface defines its vector type as that of single precision. The load reads sixteen 16-bit values and converts them to sixteen 32-bit ones, all operations execute in single precision, and the store converts them back. The data stay in half precision in memory and become single precision only for the duration of the computation (figure 4.2).

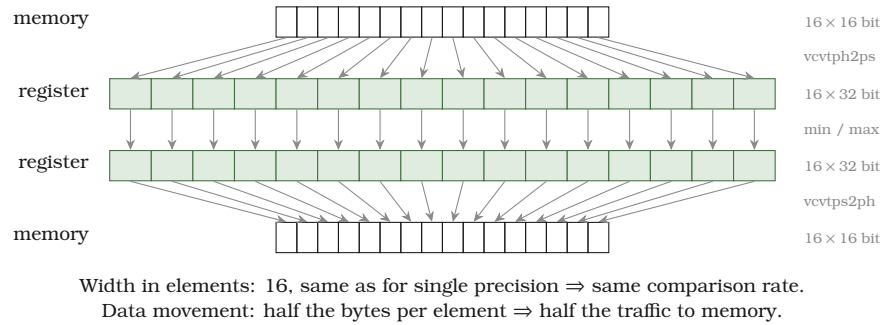


Figure 4.2: The half-precision path. The data remain in 16 bit in memory, where they occupy half the space, and are converted to single precision only within the registers, for the duration of the computation.

Every half-precision value is represented exactly in single precision, and the minimum/maximum operations simply select one of the two inputs, with no approximation or rounding involved. The result is identical to that of an implementation that would work natively in half precision.

The register width in elements remains sixteen, the same as for single precision, so the rate of comparisons does not change. Data traffic, however, is halved, and each element occupies half the bytes. For memory-bound algorithms this translates directly into a speed-up.

4.1.4 Parallelisation across multiple threads

The two algorithms are parallelised in fundamentally different ways, and the difference determines how aggressively each one scales.

Bitonic sort has global dependencies. Each group of layers needs the result of the previous one to be complete. Parallelism is possible only within the groups, with a synchronisation barrier between them.

Map-reduce has a different structure. The threads are entirely independent on disjoint segments and meet once, at the merge. No synchronisation is required during the scan, but each thread keeps its own heap of size k , and the final merge is serial, proportional to the number of threads.

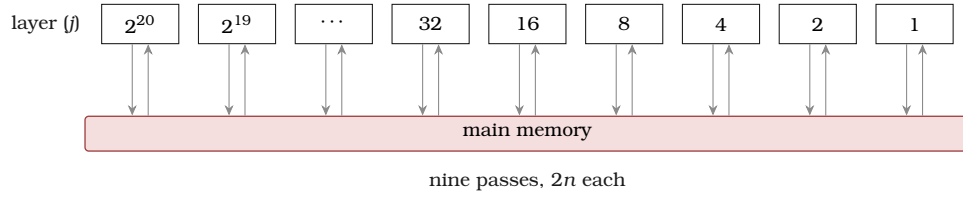
The optimal number of threads therefore differs. Bitonic sort tolerates more of them on smaller inputs; map-reduce needs a much larger share per thread to amortise the cost of the extra heap and the serial phase. In both cases the count is limited dynamically according to the input size.

4.2 Bitonic sort

The layer schedule of section 2.2.2 is built once and is shared by all three architectures. However, translating it directly into code gives an implementation that is hopelessly memory-bound. For k equal to 256, the first phase alone has 36 layers, 36 separate passes over the entire array. On an input of four million elements this means dozens of consecutive sixteen MB reads and writes, with minimal computation in between. The implementation is organised around avoiding this pattern.

The basic technique is fusing consecutive layers into one pass. A layer is characterised by the distance j between the elements it compares, and this determines what is feasible. For j smaller than the register width, the two elements of each comparison are already in the same register. A whole sequence of such layers then executes as one load, a series of shuffles and comparisons, and a store. For larger j , but a working set that fits in cache, the sequence runs chunk by chunk, staying there for all the layers. Only layers with very large j need a separate pass. A truncation layer with a distance equal to the register width also absorbs the re-sort layers that follow it (figure 4.3). When the parameters of a layer fall outside the supported range, the same sequence runs in scalar instead of vector form. The output range of each group is divided among the threads, with boundaries aligned to the chunk size or the cache line, so that two threads never write to the same line.

(a) naive transfer



(b) layer fusion

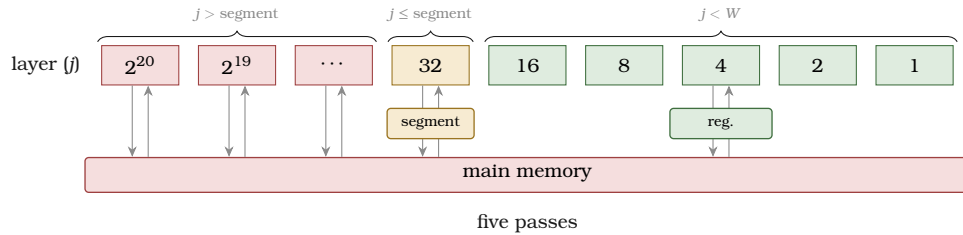


Figure 4.3: The fusion of consecutive layers into a single pass; each pair of arrows is one read and one write of the entire array. In (a) each layer is a separate pass to memory; in (b) the layers are grouped by distance j and only the last group retains a separate pass. The numbers in the figure are illustrative.

The resulting traffic is predictable from the structure of the schedule. The first phase collapses into one in-place pass, that is, one read and one write per element, $2n$ in total. Each step of the second reads n_i elements and writes $n_i/2$, and the re-sort that follows adds another $2 \cdot n_i/2$, a total of $2.5 n_i$. And since n_i halves at every step,

the sum is a geometric series:

$$\sum_i 2.5 n_i = 2.5 \left(n + \frac{n}{2} + \frac{n}{4} + \cdots \right) = 2.5 \times 2n = 5n.$$

About $7n$ elements in total. This is the origin of the repetition factor of section 3.3.4. For $n = 2^{22}$, k equal to 256, this gives 117 435 392 bytes, exactly the measured value.

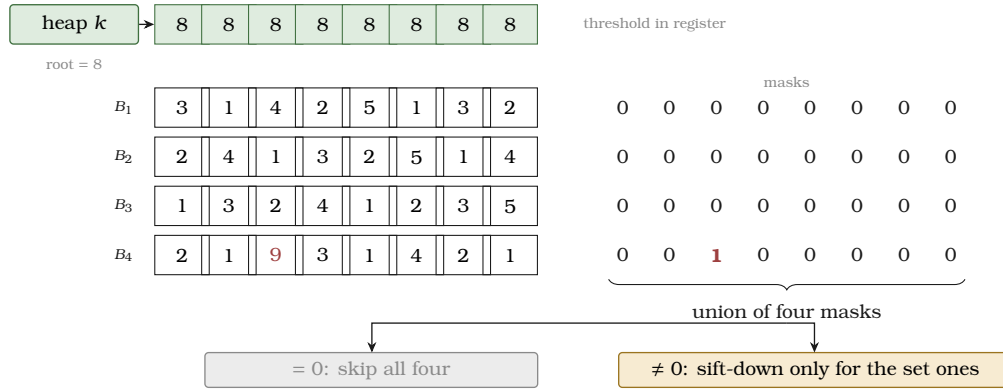
The truncation layers do not run in place, since each output element results from two input elements at different positions. A second array of the same size is maintained, with the two swapped after each such group. The extra memory is the price for avoiding intermediate copies.

4.3 Map-reduce with local heaps

Here an entirely different logic is followed. Instead of a fixed sequence of comparisons, the k best elements examined so far are maintained and every new one is tested against them. The input is split into segments, one per thread, and each thread produces its own k best. The partial results are merged at the end.

Each thread maintains a heap of size k , with the worst of the selected elements at the root. Each heap is bulk-initialised from the first k elements of the segment, which costs linearly and is cheaper than k successive insertions. From then on, the root acts as a threshold: any element that does not exceed it can be rejected with no further work.

This rejection happens in bulk. The threshold is broadcast into a vector register and compared simultaneously against an entire block of elements, producing a binary mask of candidates. The implementation examines four such blocks together and, if the union of the four masks is zero, skips all of them with one check. When the heap has already filled with large values (the usual case), most of the input passes through exactly like this: read, compared and rejected, without any element being examined individually (figure 4.4).



In the common case, the left path is followed almost always.

Figure 4.4: Bulk rejection in map-reduce. The root of the heap is broadcast across an entire vector register and compared simultaneously with four blocks of the input, producing one binary mask per block. The union of the masks selects one of the two paths at the bottom of the figure.

Scalar work runs only for the bits that are actually set. The root is replaced and the heap is restored by sifting down. Replacing the root is preferred over extraction and reinsertion, since the latter would traverse the heap twice instead of once.

This is where the data dependence of section 3.7.2 comes from. On descending input the heap fills immediately with the largest values, the threshold rises to its maximum from the start and all the rest of the input is rejected in bulk. On ascending input the opposite happens: every new element exceeds the root, no mask is zero and every element forces a full sift-down. The same code, on an input of the same size, thus performs almost no work beyond reading in one case and the maximum possible in the other, the only difference being the order in which the values appear.

The merge is simple and serial. The elements of each local heap are offered in turn to the first one, with the same threshold criterion, and the final result is sorted. The cost is proportional to the product of threads and k and independent of the input size, hence negligible for large n .

Data traffic follows directly. Every input element is read exactly once, and the merge moves twice the product of threads and k . The comparison count, however, is reported as a lower bound, since the depth of each sift-down depends on the values. The count records one comparison per element and ignores the restoration of the heap.

The thread count is limited far more conservatively than in bitonic sort. It needs about one million elements per thread (a limit found empirically), because every additional thread adds a heap of size k and burdens the serial merge. For four million input elements, four threads are used rather than eight, even when eight are requested.

4.4 Memory management and data traffic

The two algorithms solve the same problem on the same machine, but with entirely different memory behaviour. Most of the performance difference comes from there.

The fundamental difference is, first, the volume of traffic. From the expressions of sections 4.2 and 4.3, bitonic sort moves about $7n$ elements, while map-reduce moves n plus a term proportional to threads times k , negligible for small k . For an input of four million integers, the first moves 117 MB and the second 17 MB.

The nature of the access matters just as much. Map-reduce reads the input and never touches it again, purely sequential reading, the most favourable pattern for the prefetcher. Bitonic sort, by contrast, writes on every pass, with comparisons between elements a variable distance apart, at some layers larger than what fits in cache.

The footprint also differs. Bitonic sort needs a second array equal in size to the first, $2n$ elements in total. Map-reduce allocates only the local heaps, that is, threads times k elements, independent of the input size. For small k this is small enough to stay permanently in the fastest levels of the hierarchy, with the only traffic to main memory being the reading of the input itself.

Relative to the ceilings of section 3.4, the two algorithms thus end up at different points. Both perform few operations per byte, so neither approaches the compute ceiling. Map-reduce achieves a substantially higher transfer rate, as expected from its more favourable access pattern. Bitonic sort pays twice, because it moves a sevenfold volume and at a lower rate.

This advantage, however, does not hold across the whole range of k . The term of the local heaps grows with k and for the largest values stops being negligible. The heap no longer fits in the fastest cache levels, and the serial merge acquires a weight comparable to the scan. Bitonic sort, with traffic that depends much more weakly on k , has no equivalent penalty.

4.5 Reference implementation

The baseline for the processor is `std::partial_sort` from the standard library (section 3.5). It is integrated into the same harness, the same measurement loop and the same correctness check as the two implementations of this work. As a result, the times are directly comparable.

There is, however, an asymmetry that affects the speed-up measurements. The library function is strictly single-threaded; it accepts no thread-count parameter and uses only one core. In contrast, bitonic sort uses up to eight threads and map-reduce four or eight, depending on the input size. Any difference in performance between the implementations of this work and the library is therefore due to two factors at once: the algorithmic differences and the unequal number of cores.

The practical speed-up over the standard solution the programmer already has available remains, however, the central question, and that solution is indeed single-

threaded. The reader should bear in mind that a stricter thread-for-thread evaluation would give smaller speed-ups.

4.6 Limiting factors

The limiting factor is not the same across the whole range of k . In fact, this shift is so decisive that it changes which of the two implementations is faster.

For small k and large input, both implementations are clearly memory-bound. At the largest input examined, bitonic sort reaches four-fifths of the measured bandwidth ceiling and map-reduce three-quarters, with compute-ceiling utilisation around five percent for both. The limitation here is memory.

This diagnosis sets the bounds of any possible improvement. Bitonic sort does not execute its comparisons inefficiently. It is slow because it moves a sevenfold volume, and no optimisation of the kernel would speed it up substantially. Map-reduce already reads every element exactly once, close to the rate at which memory can feed it, essentially optimal, in the sense that it cannot get faster without reading less.

For large k the picture changes radically and neither implementation approaches either ceiling. At the largest value of k examined, utilisation falls to figures on the order of one percent for map-reduce and a little above ten percent for bitonic sort. When a measurement is far from both ceilings, the real limitation shifts from bandwidth or compute capability to latency and insufficient parallelism.

As for map-reduce, as k grows the heap's threshold falls and the rejection rate collapses. The fast path that skips four blocks stops activating and almost every element forces a full sift-down. At the same time the heap stops fitting in the fastest cache levels, so every sift-down becomes a sequence of random-address accesses, of depth logarithmic in k . The algorithm thus turns from bandwidth-bound to latency-bound, and at the largest value of k it ends up slower than bitonic sort.

A secondary but real factor is the thread-scaling limits. Because map-reduce needs about one million elements per thread, at medium input sizes only part of the available cores work. Part of the gap from the ceiling there is simply due to the whole processor not working.

Chapter 5

GPU implementation

5.1 Design choices

Both algorithms here face two basic challenges: the handling of data types and the distribution of the workload across the device's execution resources.

Type handling follows the processor's philosophy in a simpler form. Specifically, a structure per type exposes all the operations the algorithms need (comparisons, finding the minimum/maximum), allowing the main code to be written only once, regardless of type. The distinguishing difference from the processor is that on the graphics unit there is no question of selecting a width dynamically at run time. The compiler directly generates optimal code for the device's specific architecture, so the abstract interface serves exclusively to distinguish types.

In addition, an explicit mapping between the types of the host and the device is required, given that half precision is represented by a different type in the two environments. This mapping is implemented centrally at a single point, remaining completely transparent to the algorithms, which are written so as to be unaware of the environment in which they run.

5.1.1 Mapping onto the grid and thread blocks

The device executes threads organised into blocks, and the blocks into a grid. The distinction is essential, because threads of the same block share fast memory and synchronise with one another, whereas threads of different blocks lack such capabilities. Each algorithm must therefore place its dependencies within blocks, wherever this is possible.

Both implementations use 256 threads per block, a number sufficient to cover memory latency without limiting the number of blocks that coexist on each multiprocessor.

The grid size arises differently in the two algorithms. In bitonic sort it is determined strictly by the data. Each block takes on one tile, so the number of blocks matches the number of tiles into which the active segment is split. The mapping is absolute, with no freedom of choice. In map-reduce, by contrast, the grid is a design decision,

constrained by the occupancy the hardware allows, the heap-filling condition and the available workspace (section 5.3.1).

Regardless of the grid size, the order of memory access is decisive. Simultaneous accesses by neighbouring threads are coalesced into single transactions only if they concern neighbouring addresses.

5.2 Bitonic sort

The layer schedule is the same as in section 4.2; after all, it is built by shared code. Only the way of executing it changes, but the problem stays the same. There are many layers, and the naive mapping of one layer to one kernel launch would mean dozens of successive passes over global memory.

Here a second cost is added that does not exist on the processor. Every kernel launch has a fixed overhead, which hundreds of launches pay hundreds of times. The entire network is therefore captured once as an execution graph, instead of being launched kernel by kernel, and is replayed. Scheduling thus happens once and only once for the whole network.

The graph is composed of three kinds of kernels, with the choice determined by the distance j of the layer, as on the processor. When the distance is small, an entire sequence of layers runs inside the shared memory of a thread block (section 5.2.1). When it is too large to fit in a tile, the data stays in global memory and one kernel executes up to five successive layers, keeping the intermediate results in registers. The truncation layers have their own kernel, which reads two elements and writes one to a second array, with alternation as on the processor.

5.2.1 Organisation into tiles and shared memory

The shared memory of a thread block is explicitly managed, unlike the processor's automatic cache. The implementation loads each block's tile into it (section 5.1.1), executes as many layers as possible entirely within it and writes it back once. No additional traffic to global memory is needed for the layers fused this way.

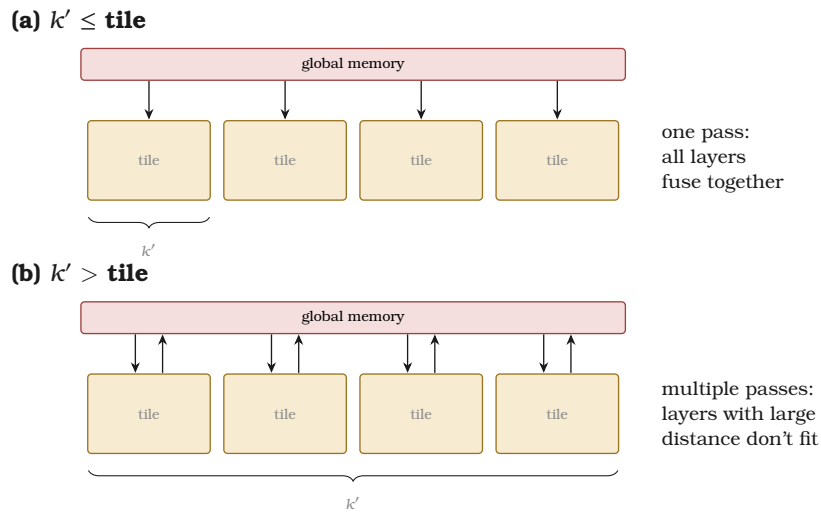
For 32-bit integers the tile ends up at 8192 elements, a size arrived at after testing larger and smaller tiles, around two constraints: it does not exceed 48 KB per block without special configuration, and it shrinks when the working set fits in the second-level cache, where the bottleneck shifts from traffic to main memory to the number of simultaneously active blocks.

This constraint is inherent to the architecture. Published partial-sorting methods for graphics units are subject to the same bound, limiting k to a few hundred, since their structures must fit in shared memory and in registers [3, 5].

A layer is fused only if the distance between the elements it compares fits in the tile, smaller than half its size. The distances of the first phase reach half the size of the sorted blocks, so the critical size is the k' of section 4.2. As long as the blocks fit in the tile, almost all layers are fused (figure 5.1).

For small and medium values of k this holds, and the data traffic turns out essentially the same as on the processor (about seven times the naive estimate), with the two architectures agreeing down to the last byte for k up to 4096 (section 3.3.4). They have the same schedule, an equivalent fusion strategy, only explicitly managed memory instead of automatic.

At the largest value of k the condition breaks down. The sorted blocks reach 131072 elements, sixteen times the tile. Most of the layers of the first phase are no longer fused and execute as passes over global memory. The measured traffic rises correspondingly, from sevenfold to between twenty-two and twenty-nine times the naive estimate, depending on the input size.



Measured traffic goes from $7n$ to between $22n$ and $29n$ once k' exceeds the tile.

Figure 5.1: The relationship between the sorted block and the shared-memory tile. A layer can execute without returning to global memory only if the distance between the elements it compares fits in the tile. As long as sorted blocks of size k' fit in it (a), nearly all layers fuse into one pass and data traffic matches that of the processor. Once k' exceeds the tile size (b), layers with large distance execute as separate passes over global memory and traffic multiplies. For 32-bit integers the tile holds 8192 elements, so the transition falls between the two largest values of k examined.

5.3 Map-reduce with local heaps

The algorithm is the same as in section 4.3. Each thread keeps a heap of size k , where the root acts as a threshold and the partial results are merged at the end. The difference is the scale of the decomposition.

On the processor the threads are few, each with a contiguous segment of the input. On the graphics unit thousands of threads stride through the input with a stride equal to their total number, so that neighbouring accesses are coalesced into single memory transactions.

Each thread keeps its own heap. The heaps do not fit in shared memory, because the product of threads and k far exceeds the available 48 KB, and they are kept in a workspace in global memory. This is where the implementation diverges from the processor with respect to data traffic. With thousands of threads, the term that was negligible there no longer is. Because each thread requires a heap of size k , the total number of threads is subject to an upper memory bound, binding for large k .

The merge happens at two levels. Within each block the heaps are combined pairwise in a tree, with a number of steps logarithmic in the block size. The result of each block is then merged with the rest. This structure avoids the serial phase of the processor, but introduces its own cost, since every step of the tree reads and writes heaps from global memory.

5.3.1 Parallelism limit for large k

The constraint is binding where the graphics unit would be most useful. At the largest value of k and an input of sixteen million elements, the ratio n/k is only 128. A device with thousands of cores can effectively occupy one hundred and twenty-eight threads, a tiny fraction of the available threads.

All effectiveness depends on rejection. Every element that does not exceed the root costs one comparison. The root, however, becomes meaningful as a threshold only after the heap has filled. With fewer than k elements inside, every new one is inserted regardless, without any rejection.

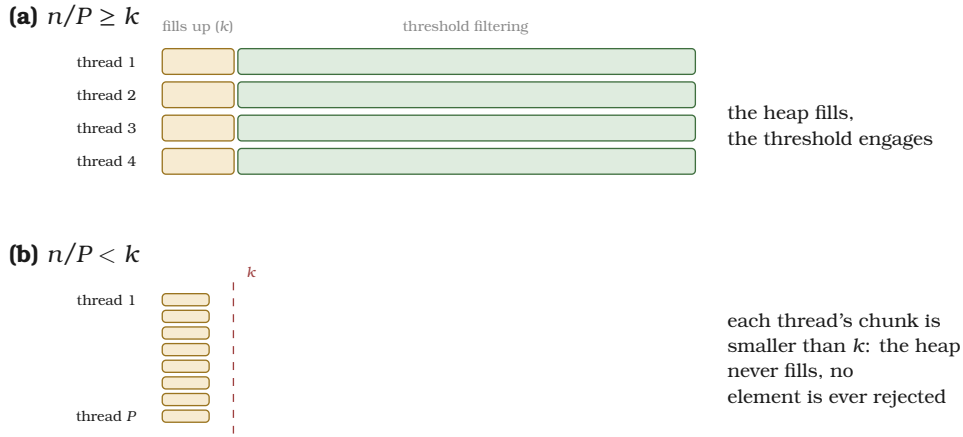
This gives rise to the exact constraint. If an input of size n is split among P threads, each thread sees n/P elements and its heap fills only if $n/P \geq k$, that is,

$$P \leq \frac{n}{k},$$

the upper bound on the number of threads that contribute substantially. Beyond it, the extra threads do not speed up the computation; they burden it. They fill the workspace with heaps that never filter and add levels to the merge tree. No code avoids this underutilisation, since it stems from the very way the algorithm works.

The implementation reacts to this condition. Before launch it computes whether the heaps manage to fill under full occupancy of the device. If so, the number of blocks is deliberately limited, inversely proportional to k , so that each thread takes on a larger share and its heap filters efficiently. Nominal parallelism is sacrificed for actual rejection. When the heaps are not going to fill in any case, the reduction would offer nothing and the device works at full occupancy (figure 5.2).

In every run the stricter of the constraints prevails. The memory limit of section 5.3 acts alongside this one, together explaining why the implementation is unable to make use of the device at large k .



Useful parallelism is therefore bounded by $P \leq n/k$, regardless of grid size.

Figure 5.2: The parallelism limit of map-reduce for large k . Each thread maintains its own heap of size k and can reject elements only after the heap fills, i.e., only after it has seen at least k elements. When the chunk assigned to each thread is larger than k (a), the threshold engages early and the rest of the chunk is filtered. But when the number of threads increases enough that the chunk becomes smaller than k (b), no heap ever fills and filtering never engages. This is a property of the algorithm, not the implementation: neither the grid size nor the workspace layout can remove it.

5.4 Memory management and data traffic

The graphics unit is characterised by two separate memory systems connected by a bus incomparably slower than either. The device's local memory delivers about 320 GB/s, the bus to the host only 14 GB/s, a difference of more than twentyfold.

Within the device the picture resembles that of the processor. For small k , bitonic sort moves the same sevenfold volume. The traffic of map-reduce, however, depends noticeably on the input size, due to the per-thread workspace (section 5.3).

It is the transfer to and from the device, however, that dominates. For four million integers, the roughly seventeen MB take more than a millisecond just to arrive, while the bitonic sort kernel itself finishes in half that. The transfer costs more than twice the computation it serves.

This work measures two scopes (section 3.3.1). A comparison in the algorithmic scope alone would give the graphics unit an advantage it does not have when the problem is posed as a transfer from main memory and back.

The established technique for improving transfers (pinned memory on the host to avoid an intermediate copy) offers no substantial benefit here. The two paths differ by about two percent, an indication that the limit is set by the bus itself.

As for the footprint on the device, the two implementations follow the logic of the processor. Bitonic sort allocates two arrays of size n (section 5.2) and map-reduce the per-thread heaps (section 5.3). For large k , the heap workspace can approach the size of the input itself.

5.5 Reference implementation

The baseline is the top-k selection function of CUB, which ships with the CUDA toolkit. On the processor the reference implementation was single-threaded and the comparison conflated two causes. Here the library runs on the same device, exploits the same parallelism and is measured the same way. The comparison is therefore rigorous.

The integration follows the library’s two-call protocol. The first declares the required auxiliary space without computing anything, the space is allocated, and the second performs the selection. Only the second is bracketed by the events that define the algorithmic scope. Allocation and transfers stay outside, as in the two implementations of this work.

The counted traffic and number of comparisons are reported here too as a lower bound, for the reason of section 4.5. Only input and output; the internal passes are not observable.

On the largest input examined the library completes in nearly constant time as k varies over four orders of magnitude, while the two implementations of this work are burdened significantly. Only for the smallest k does map-reduce overtake it.

5.5.1 Radix selection in CUB

The library [14] uses radix selection (section 2.2.4). It never sorts the entire input; it performs a few passes, each a sequential read with frequency counting, exactly the pattern the architecture executes best. Moreover, the number of passes is determined by the width of the data type. Selecting eight elements costs essentially as much as selecting one hundred and thirty thousand.

Independence from k is the fundamental difference from both algorithms of this work. Bitonic sort is burdened as k grows: the sorted blocks grow and stop fitting in the tile (section 5.2.1). Map-reduce is burdened even more (section 5.3.1). Radix selection has no equivalent collapse point. It is not, however, indifferent to the data, in the sense of section 2.2.4. It depends on the distribution of digits in the values.

5.6 Limiting factors

Here the two measurement scopes mean different limiting factors. In end-to-end, the bus to the host dominates (section 5.4). What follows concerns the algorithmic scope, where the limits of the device itself show.

For small k , bitonic sort is clearly memory-bound, reaching about three-fifths of the measured bandwidth ceiling, with utilisation of the compute ceiling of only two and a half percent. It exploits the device efficiently; it simply has to move more volume.

The reference library makes the distinction quantitative, and it is the most instructive point of the chapter. It exploits only about one-fifth of the bandwidth ceiling, clearly less than bitonic sort, and yet finishes faster. Proximity to the ceiling measures the

quality of the implementation, while the total volume of traffic measures the quality of the algorithm.

Map-reduce for small k is the only point where an implementation of this work outperforms the library. It reads the input once and rejects almost everything, approaching the minimum possible work. Ceiling utilisation stays modest, pointing to room for improvement in the organisation of accesses.

For large k , map-reduce collapses to the point where utilisation of both ceilings becomes a fraction of one percent, for lack of work to perform. As explained in section 5.3.1, at the large values of k the overwhelming majority of threads fail to fill their local heap in time, leaving most of the device idle.

Bitonic sort is burdened far more mildly. Bandwidth utilisation drops to about one-third, as the loss of fusion in shared memory increases traffic without a corresponding increase in useful work. It stays functional, and at the largest k it ends up two orders of magnitude faster than map-reduce.

Neither of the two is ultimately limited by something that further code optimisation would fix, since the obstacle lies in the algorithm itself. Bitonic sort would benefit only from less traffic, that is, a different network, while map-reduce only from a different decomposition for large k , so that parallelism does not depend on the ratio n/k .

Chapter 6

NPU implementation

6.1 Design choices

The neural processing unit is a radically different architecture from the two previous ones. It does not execute a program in the usual sense, because it is an array of tiles configured in advance, with no cache, no dynamic memory allocation and no ability to change its dataflow at run time.

On the processor and the graphics unit the question was how the algorithm is distributed across threads. Here the question is what small, fixed piece of work the array executes efficiently and how the host feeds it without interruption.

6.1.1 Development flow with MLIR-AIE

There is no programming environment equivalent to that of the graphics unit. This work relies on MLIR-AIE [13] and the IRON interface [20]. Development requires two separate artefacts, from different toolchains, which are combined at the end.

The first describes the array itself: which tiles are used, which channels connect them, with what pattern data moves to and from main memory. It is written in a domain-specific language embedded in Python, translated into an intermediate representation and finally into a binary hardware configuration file. The second is the compute kernel of each tile, written in C++ with the architecture's vector primitives and compiled separately.

The configuration is fixed at compile time. The topology, the sizes of the intermediate buffers, the transfer pattern, are all decided before each run, independently of the parameters of the problem. Changing k or the input size changes nothing on the device, only the way the host splits the input and interprets the results.

6.1.2 Tile topology and object_fifo

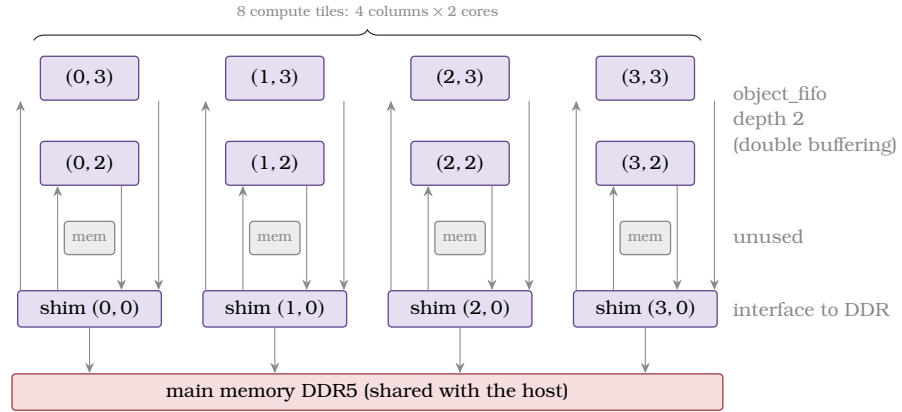
The array is organised into columns. Each column has a shim tile that communicates with main memory and, above it, compute tiles.

Data transfer is not programmed with explicit copy instructions; it is declared as a channel between two tiles. Each channel is backed by a number of intermediate

buffers, here two, so as to allow overlap: while the tile processes one, the direct memory access mechanism fills the other. Without this, every tile would stall on every transfer. This technique is known as double buffering.

Each compute tile has its own pair of channels to the shim tile of its column, one for input, one for output. There is no communication between compute tiles, which is a deliberate choice. The work assigned to them is by definition independent per tile and no exchange is required.

The amount of data per dispatch follows from the product of the above (figure 6.1). Each tile receives chunks of 1024 elements, 128 such chunks per dispatch, so the batch size scales with the number of tiles the algorithm in question uses. Larger inputs are split into successive batches.



Per core: 128 chunks $\times$ 1024 int32 elements. Per batch: $8 \times 128 \times 1024 = 1\,048\,576$ elements.

Figure 6.1: The tile topology of the implementation and the object_fifo queues that connect them. Each column has one interface tile (shim) that communicates with main memory and two compute tiles, each of which connects to its column's shim through a pair of depth-two queues: one for input and one for output. The depth of two allows the core to process one chunk while the next is transferred. The memory tiles in the middle row are unused, since each core consumes a portion of the input disjoint from the others and no sharing or redistribution of data between them is required.

6.2 Bitonic sort

An essential difference affects all the measurements: the array departs from the truncated bitonic sort schedule of section 4.2 for purely structural reasons. Specifically, the schedule requires comparisons between elements arbitrarily far apart, which presupposes data exchange between tiles and multiple passes over the input. The topology of section 6.1.2 lacks these mechanisms, since the tiles work independently and every element passes through the system exactly once.

The problem is therefore reframed. The array takes on a fixed, local task. It independently sorts successive sequences of sixteen elements, and the host takes on the selection, merging these sorted sequences into a heap of size k .

Because each sequence of 16 elements comes back sorted, the host can stop examining it as soon as it encounters an element that does not exceed the root of the heap. Because of the sorting, it is guaranteed that all the following elements of the same sequence will be equally or more unsuitable. The sorting on the array is not an end in itself, but a mechanism for early rejection.

6.2.1 Compute tile kernel

Although an official reference implementation of bitonic sort is provided for the AI Engine [17], that example is limited to a full sort inside a single tile.

The size sixteen is dictated directly by the hardware. It corresponds exactly to the number of 32-bit integers that fit in a vector register of the architecture. Sixteen elements with a bitonic network need ten compare-exchange steps, each entirely within the register.

Each step follows the same motif as in section 4.1.1. A shuffle brings each element next to its partner, the minimum and maximum are computed, and a mask selects which one each position keeps. The entire network runs without any access to the tile's memory. The data are loaded once, undergo ten transformations in registers and are stored once.

Each invocation processes a chunk of 1024 elements, sixty-four independent sequences. There is no dependency between them, which lets the compiler pipeline them and hide the latencies of the individual instructions.

6.2.2 Scaling to multiple tiles

The sequences are independent, so scaling to more tiles is trivial. Each one takes on a different chunk of the batch, without synchronisation or data exchange. Only the transfer pattern in the device configuration changes.

The implementation was extended to use two compute tiles per column instead of one, eight in total. The extension yielded a noticeable improvement, confirming that up to that point the implementation had been limited by the computation on the array. More importantly, it moved the limiting factor elsewhere. With eight tiles the array stopped being the bottleneck and the time is now spent almost entirely on the host. Scaling to more tiles would no longer change anything.

6.3 Map-reduce with local heaps

The second algorithm follows the same work-distribution principle, with an even more limited role for the array. Four compute tiles are used, one per column, and the kernel does not sort. For each input chunk it computes one summary, the best value it contains.

The host keeps the heap of size k and uses the summaries as a filter. If the best value of an entire chunk does not exceed the root, the chunk is rejected as a whole, without any of its elements being examined. Only the chunks that pass the filter are examined element by element.

The filter presupposes a realistic threshold before the first chunk. On the processor and the graphics unit this comes about automatically, since the heap is bulk-initialised from the first k elements of the input. In the case of the array, the host samples in advance a portion equal in size to the larger of k and two percent of n , and inserts it directly into the heap before the first job is dispatched, so that the filter works effectively from the start.

The array produces the information that lets the host perform the selection cheaply, which suits its nature as fixed, local and fully predictable for operations on a data stream.

6.4 Memory management and data traffic

The unit has no memory of its own, and the data do not cross a bus to a separate device, as with the graphics unit. They already reside in the same physical memory as the host's, and what is called a transfer is in reality the synchronisation of a shared buffer.

As far as transfer cost is concerned this is an advantage; there is no bandwidth gap as in section 5.4. There is, however, a reverse side. Because the memory is shared, every byte the host prepares for the array consumes the same bandwidth it would use if it performed the computation itself.

6.4.1 Pipelining and the role of the host

Less than one tenth of the time is spent waiting for the array. The remaining ninety-two percent is work the host itself does, as the measurement of the individual stages shows, perhaps the most illuminating result of the chapter.

So that the array does not sit idle between successive batches, execution is organised as a three-stage pipeline, with two sets of buffers that alternate. While the array processes one batch, the host prepares the next and merges the results of the previous one (figure 6.2). For an input of sixteen million integers (sixteen batches), averaged over fifteen repetitions, the time is distributed as in table 6.1.

Stage	Time (ms)	Percentage
Batch preparation	4.84	58 %
Result merging	2.07	25 %
Waiting for the array	0.69	8 %
Dispatching work	0.10	1 %
Initialisation	0.08	1 %
Other	0.56	7 %
Total	8.34	100 %

Table 6.1: Time breakdown of the offload region by stage. The “Other” row covers the portion of the measurement not attributed to any of the other stages, consistently present in every repetition.

A pipeline performs as well as its slowest stage, and the slowest stage is not on the

device.

The measurement of the algorithmic scope here must not be read as the compute time of the tiles (section 3.3.2). It encloses the entire offload procedure, most of which executes on the processor.

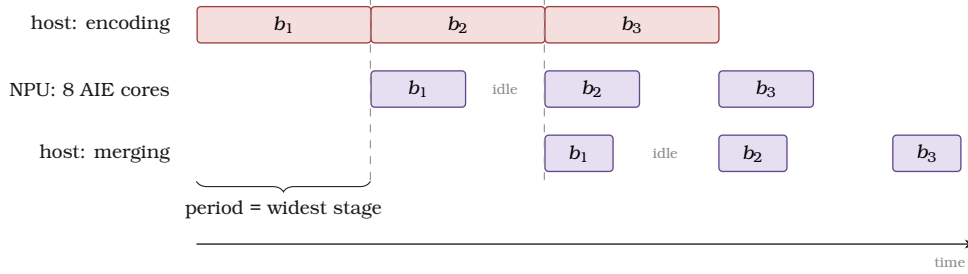


Figure 6.2: The three-stage pipelining of the offload. While the tiles process batch b_i , the host is already encoding b_{i+1} into the second buffer and merging the results of b_{i-1} . Because the three stages overlap, the total time is not their sum but the product of the number of batches and the duration of the widest stage. In this implementation that stage is host encoding: only it executes without interruption, while the other two remain idle for part of each period. No further speed-up of the AIE core can therefore reduce the total time.

6.4.2 Data encoding and types

The array’s kernel operates exclusively on 32-bit integers. It does not need to know what the values it compares represent. Every data type of this work is therefore mapped to an integer key that preserves the relative order of the values, following the platform’s type-conversion rules [21].

The mapping differs by type. Signed integers are used as is; for unsigned ones the most significant bit is flipped. In single precision either the sign or all the remaining bits are flipped, depending on the sign, so that the binary comparison coincides with the arithmetic one. Half precision is first widened to single (an exact conversion) and follows the path above.

The encoding runs on the host and forms part of the preparation stage of table 6.1. Handling the data type burdens only the processor computationally. In fact, the initial, scalar implementation of the half-precision encoding proved so expensive that it noticeably delayed the whole stage. For this reason, it was replaced by a vectorised path that converts eight values per instruction.

6.4.3 Final correction on the host

Double precision does not fit in a 32-bit key. The encoding keeps the 32 most significant bits of the full 64-bit key, so two different values can end up with the same key. The array returns something weaker than the exact result: a threshold.

An additional stage on the host restores correctness. From the keys the array returned, the worst one is taken, which defines the threshold. The host traverses the

entire input again, collects the elements whose key falls within the threshold and performs exact selection among them.

The cost is one extra pass over the input, which exceptionally burdens double precision. The benefit is that the result is exact and passes the same correctness check as all other data types, without relaxing the criterion. An approximate result for a single type would make the comparison across types unequal.

6.5 Reference implementation

The baseline is the same standard-library function as for the processor, run entirely on the host. The reasons for this choice were discussed in section 3.5. It is only worth recalling the essential point: there is no top-k implementation for the array itself.

6.6 Limiting factors

The limiting factor is already visible in table 6.1 and it is the host. But why is the host's work so expensive?

The preparation stage (three-fifths of the time) comprises a full read pass over the input, the conversion of each element into an integer key and a write of equal volume to the shared buffer. In total this gives data traffic at least twice the input size, over the same memory and at the same bandwidth the processor would use. The merge adds yet another read pass over the results.

Even if the array completed its own work instantaneously, the total time would drop by less than one tenth, since the time is not spent there. The speed-up therefore cannot come from the device.

The comparison with the processor sharpens the argument. The host that feeds the unit is the same processor that solves the entire problem on its own. Here it is asked for almost the same amount of data traffic, merely to prepare the data for someone else. The cost of preparation is comparable to the cost of the solution itself, which is why the offload does not pay off.

This conclusion, however, concerns this particular problem. Top-k selection has minimal operational intensity, with one comparison per element and almost no reuse. A dataflow array performs well when every element that enters it undergoes many operations, so that the feeding cost is amortised. Here the opposite happens, and this mismatch between the problem and the architecture explains the result better than any implementation detail.

Chapter 7

Results and evaluation

This chapter presents the results and interprets them in terms of the mechanisms of the three preceding chapters. Unless stated otherwise, the plots concern 32-bit integers and a uniform input distribution. The effect of data type and distribution is examined separately in sections 7.6 and 7.7.

7.1 Correctness

Every timing comparison presupposes a correct result. The main sweep covers 2700 cases: three architectures, three algorithms, five data types, input sizes from 2^3 to 2^{24} , four values of k . All were verified against the verification implementation (section 3.6) with a 100% pass rate. Every time that follows therefore corresponds to a correct set of k elements.

7.2 Execution time

7.2.1 Dependence on input size

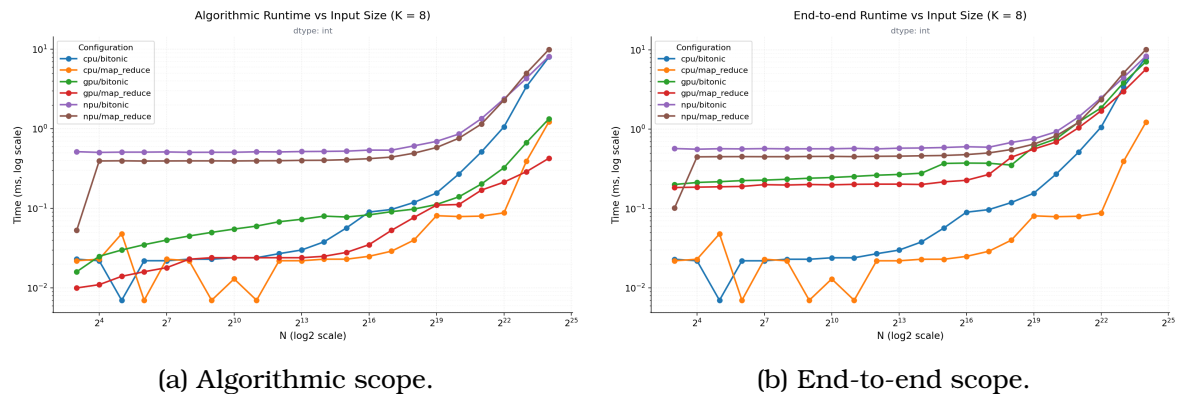


Figure 7.1: Time as a function of input size, for k equal to 8, under the two measurement scopes. The ranking of the architectures is completely overturned between the two.

In the algorithmic scope (left), the graphics unit is faster at the smallest input, 0.010 ms against 0.022 ms for the processor, and again at the largest, 0.43 ms against 1.23 ms. Across almost all of the range in between, however, the processor leads. The exact crossover point moves between repeated runs of the same sweep. At such small absolute times both architectures are close to the timer’s resolution, so there the ranking more likely reflects measurement noise.

The neural processing unit shows a clear floor at small sizes. The curves are flat at around half a millisecond up to about 2^{16} elements, dominated by the fixed cost of dispatch and waiting. The processor has an analogous but much lower floor, on the order of five hundredths of a millisecond.

Only one point stands out. At the smallest size, where n and k coincide, map-reduce on the neural processing unit drops noticeably below this floor. This is due to the sampling of section 6.3. When $n=k=8$, the sample that seeds the heap covers the entire input and no job ever reaches the tiles.

In the end-to-end scope (right), with data transfer inside the measurement, the picture is completely overturned. The processor is faster than every other architecture at *all* sizes, even the largest. The 0.43 ms of the graphics unit’s kernel becomes 5.69 ms in total, while the processor stays at 1.23 ms, since for it the two scopes coincide.

The algorithmic scope shows how well each device computes; the total answers whether it is worth assigning the workload to it. The neural processing unit needs 9.97 ms at the largest input, more than any other architecture in both scopes.

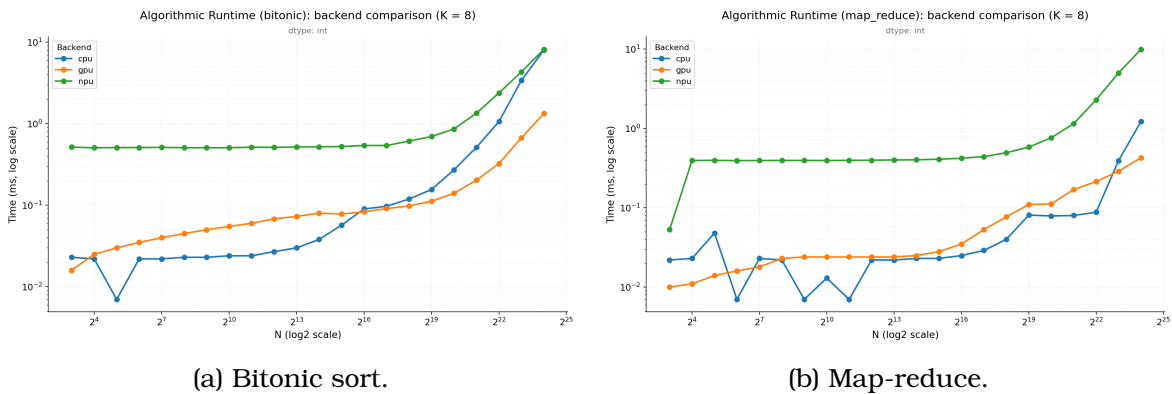


Figure 7.2: Each algorithm on the three architectures, for k equal to 8.

Comparing the same bitonic sort on the three architectures is illuminating, since it executes the same layer schedule and the difference is due solely to the implementation. The two general-purpose architectures trade places along the range: the processor is faster over most of the small to medium sizes, the graphics unit only at the largest, reaching a factor of six at the largest size, consistent with the difference in bandwidth. As above, the exact crossover point moves between runs and should not be read as a fixed threshold. The neural processing unit stays close to the processor at these larger sizes, consistent with its time being spent almost entirely on the host.

Map-reduce does not admit the same comparison in the first place, since each architecture maintains and merges its local heaps in a fundamentally different way (chapters 4 to 6). The right-hand figure shows instead how differently the same algorithmic idea unfolds in three implementations. The graphics unit is the fastest across almost the whole range, with the processor following at a roughly constant distance, except at small sizes, where the same caveat about measurement noise applies. At the largest input the graphics unit stays faster, but without the intermediate crossover of bitonic sort. The neural processing unit here does not approach the processor (in sharp contrast to its behaviour in bitonic sort) and stays about an order of magnitude slower than both at every size.

7.2.2 Dependence on the parameter k

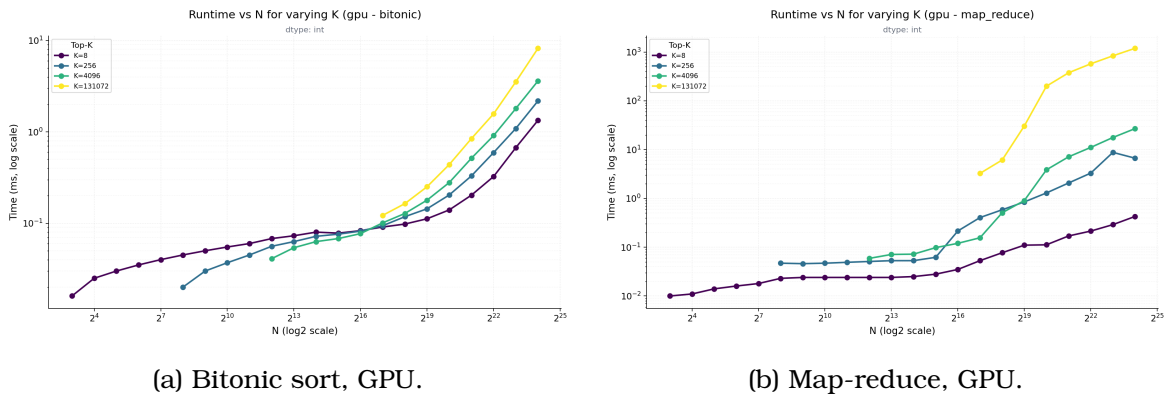


Figure 7.3: Time as a function of input size, with a separate curve for each value of k. The bitonic sort curves stay close to one another, while those of map-reduce diverge by orders of magnitude.

Implementation	k=8		k=131072	
	ms	rank	ms	rank
CPU bitonic	8.01	4	43.33	2
CPU map-reduce	1.23	2	60.37	5
GPU bitonic	1.34	3	8.22	1
GPU map-reduce	0.43	1	1191.37	6
NPU bitonic	8.17	5	56.70	3
NPU map-reduce	9.97	6	59.69	4

Table 7.1: Algorithmic time for the two extremes of k, on an input of 2^{24} elements. The ranking concerns the six implementations of this work and does not include the reference implementations.

The dependence on k is where the ranking of the algorithms is overturned. For bitonic sort the four k curves nearly coincide, since k only affects the size of the sorted blocks. For map-reduce on the graphics unit the same curves are three orders of magnitude apart.

The fastest implementation for small k becomes the slowest for large k, a difference of three orders of magnitude. Map-reduce on the graphics unit slows down by a

factor of 2800 between the two extremes; bitonic sort on the same architecture by barely six. At small k the processor's two implementations rank higher than at large k : map-reduce overtakes bitonic sort on the graphics unit, and bitonic sort overtakes that of the neural processing unit. The overturn, however, stays fixed, and the cheapest implementation for $k=8$ is never the cheapest for $k=131072$.

Independent studies on graphics units from two different manufacturers record the same dependence on k , and they too address it with adaptive algorithm switching [3, 15]. It is therefore rather unlikely to be a peculiarity of this particular implementation. There, the growth in time of partial-sorting methods with k is attributed explicitly to the $O(\log^2 k)$ complexity of the underlying bitonic network.

The cause was given in section 5.3.1. The heap filters only once it is full, and the number of threads that contribute is capped at the ratio n/k . For the sizes of table 7.1 this ratio is 128. In other words, the device effectively employs one hundred and twenty-eight threads out of the thousands it has. The processor, with only eight threads, never runs into the same limit, so its slowdown is markedly milder.

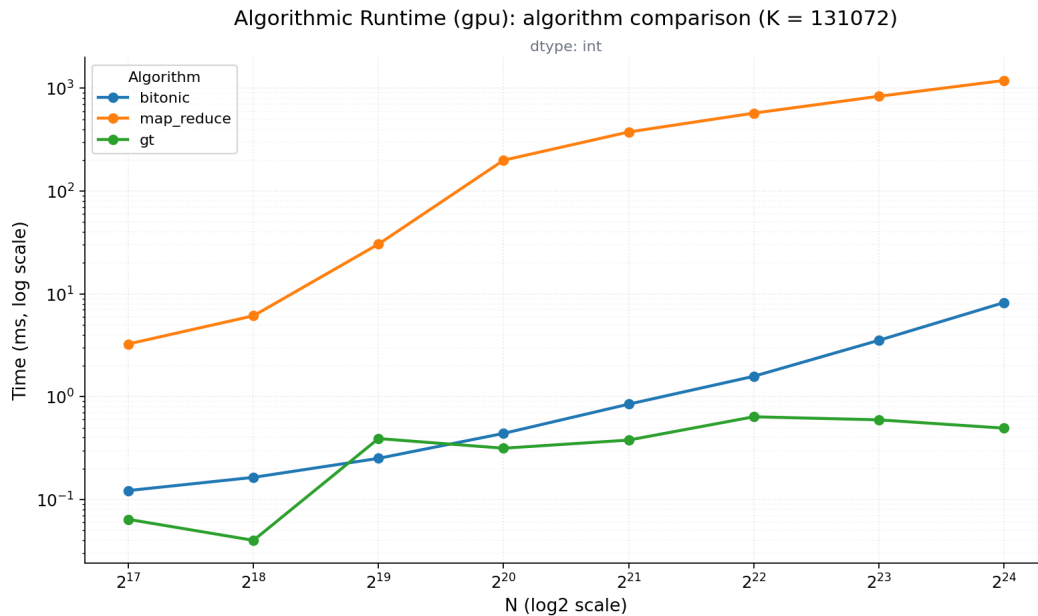


Figure 7.4: The three algorithms on the graphics unit for k equal to 131072.

There is no universally preferable algorithm. The choice between a sorting network and heap-based selection depends on the ratio n/k , not on the architecture.

7.3 Comparison across architectures

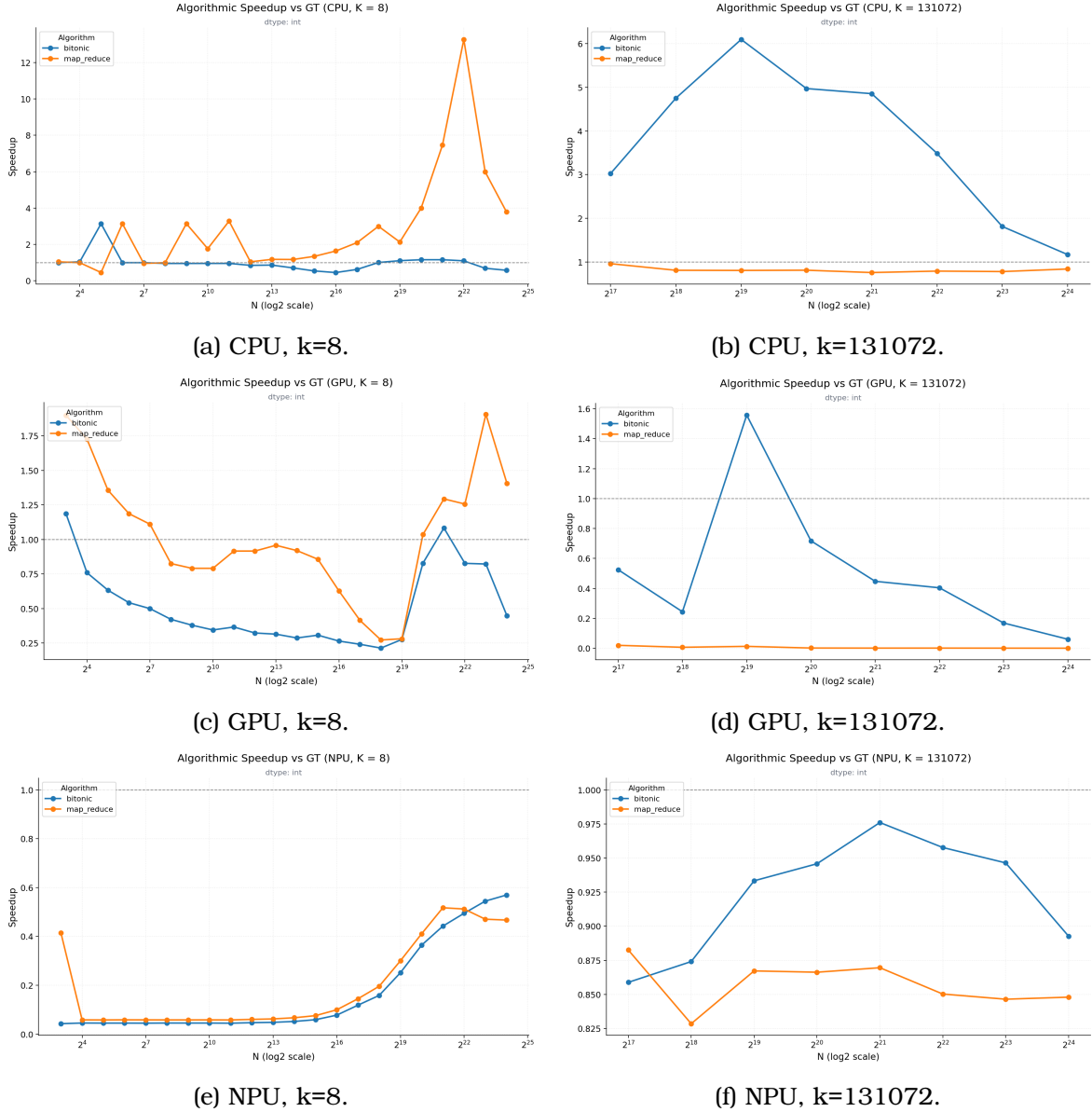


Figure 7.5: Speed-up over the reference implementation of the same architecture. Left column k equal to 8, right column k equal to 131072, one row per architecture. Values above one mean that the implementation of this work is faster than the library.

Comparing each implementation with the baseline of the same architecture neutralises hardware differences, and so gives the most honest picture.

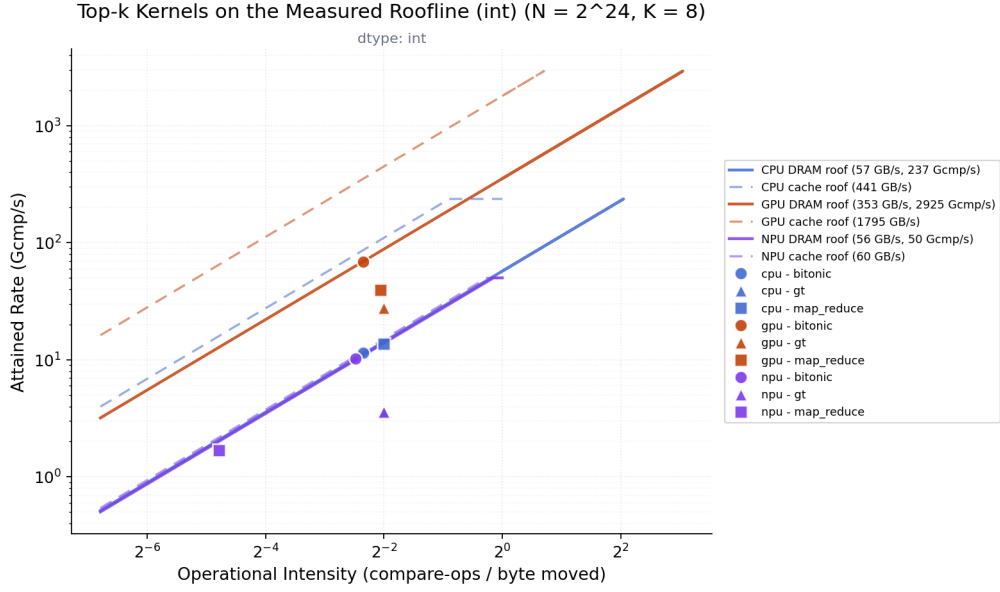
On the processor, map-reduce is faster than the library by a factor of three for small k , although the speed-up is not constant. It peaks at thirteen times for n equal to 2^{22} , where the working set (16 MB) fits exactly in the processor's L3 cache (figure 7.7), and falls back again once the input exceeds this limit. The limitation of section 4.5 still applies, though: the library is single-threaded, so the factor is not purely algorithmic.

A second notable speed-up appears elsewhere in the figure, a consequence of the overturn of section 7.2.2. At large k , where heap-based selection collapses, bitonic sort on the processor becomes about six times faster than the library, which also relies on a heap of size k and suffers the same penalty, while the sorting network stays largely unaffected. The advantage fades as the input grows further, the network's data traffic starts to dominate, and it narrows to a little above one at the largest input.

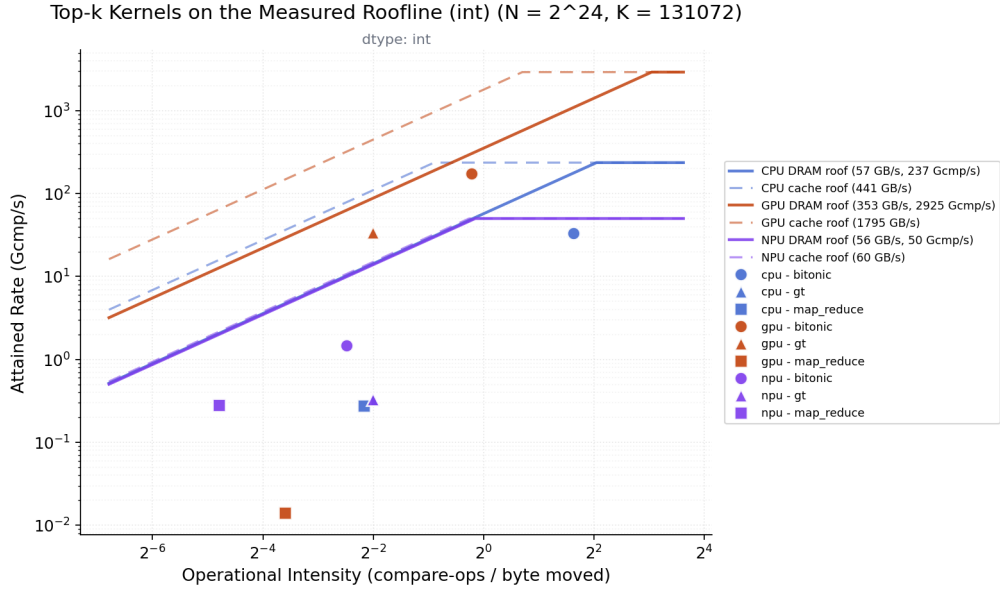
On the graphics unit the comparison is made on completely equal terms, given that the same device and the same level of parallelism are used. Map-reduce wins for $k=8$, while for large k the library is faster by more than a thousand times, since radix selection has no collapse point with respect to k (section 5.5.1).

On the neural processing unit, for the integer types the figure refers to, no implementation surpasses the library anywhere, since the same processor that would run the library is consumed in any case by preparing data for the unit (section 6.6). The qualification about integer types is deliberate. For floating point, especially half precision, the picture is overturned and the neural processing unit does surpass the library there. The explanation is given in section 7.6.

7.4 Roofline analysis



(a) k equal to 8.



(b) k equal to 131072.

Figure 7.6: Placement of the runs against the measured ceilings, for the largest input. Restricted to the maximum size, where the working set exceeds the cache and the main-memory ceiling is the relevant one.

Each run is placed here on a plane of operational intensity and achieved rate. All implementations cluster on the left, at very low operational intensity, and none approaches the horizontal part of the roof. No implementation, on any architecture, is limited by compute capability. The finding agrees with an independent measurement on data-centre accelerators, where the fastest known selection kernel uses over ninety percent of the bandwidth against about half of the compute capability, and the time scales with the bandwidth of each device [3].

Juxtaposing the two values of k highlights how sharply the algorithms differ in their use of the hardware. On the one hand, bitonic sort on the graphics unit sits consistently on the sloped part of the theoretical roof, regardless of k . This position visually confirms the finding of section 5.6. The implementation uses the available bandwidth almost fully and is limited solely by the volume of data it has to move.

On the other hand, the behaviour of map-reduce on the same architecture changes entirely. While for $k=8$ it lies near the other points, at the large value of k it collapses by three orders of magnitude, now far from both ceilings. This placement (neither on the sloped nor on the horizontal part of the plot) is the signature of a limitation independent of memory or compute. It reflects the collapse of parallelism of section 5.3.1.

This finding stems from the very nature of the problem. Because top- k selection performs practically only one comparison per element, without reuse, its operational intensity is low by definition.

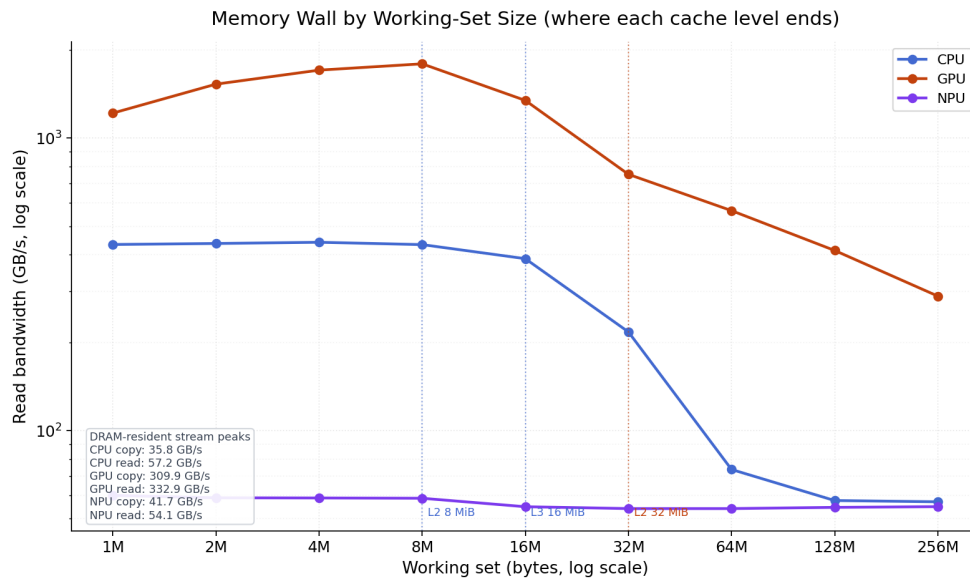
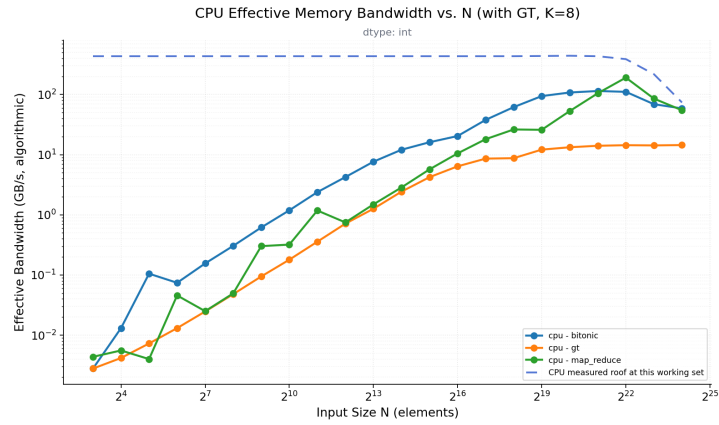


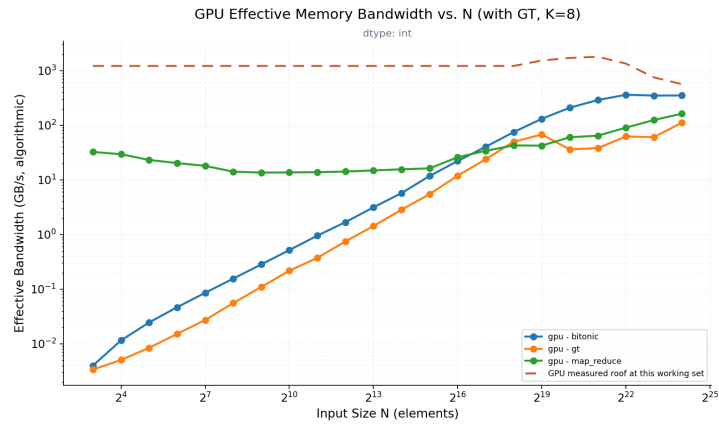
Figure 7.7: Measured read bandwidth as a function of working-set size, for the three architectures.

The maximum theoretical limit (ceiling) against which each run is compared varies dynamically, since it is a function of the working set in each case (section 3.4). Figure 7.7 shows the measured scale. The distinct levels correspond to the tiers of the memory hierarchy, while the drop at the right-hand edge marks the point where the working set exceeds the capacity of the last tier.

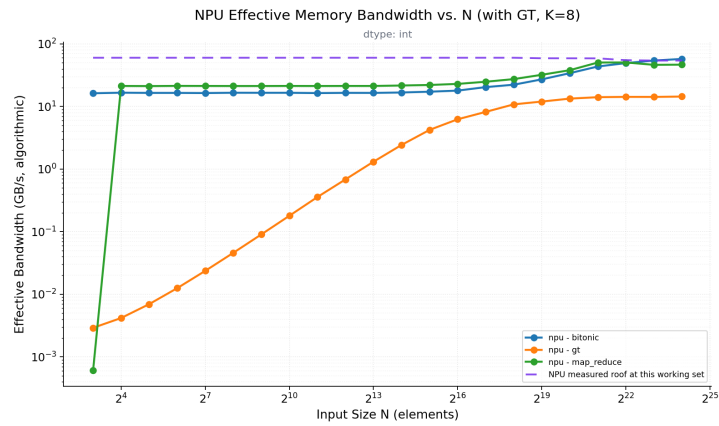
7.4.1 Memory bandwidth utilisation



(a) CPU.



(b) GPU.



(c) NPU.

Figure 7.8: Achieved bandwidth as a function of input size, one architecture per panel, with its measured ceiling drawn as a dashed line. The distance of each curve from the dashed line is the unexploited margin.

Memory-ceiling utilisation increases with input size and levels off once the working set exceeds the cache. At the largest input, bitonic sort on the processor reaches about four-fifths of the ceiling, ahead of the graphics unit's three-fifths. Compute-

ceiling utilisation stays below five percent on both.

The juxtaposition with the library on the graphics unit shows that proximity to the ceiling does not imply speed. Bitonic sort uses a clearly larger fraction of the bandwidth than the library and is nonetheless slower. The former shows how well the code is written, the latter how efficient the algorithm itself is.

7.5 Energy assessment

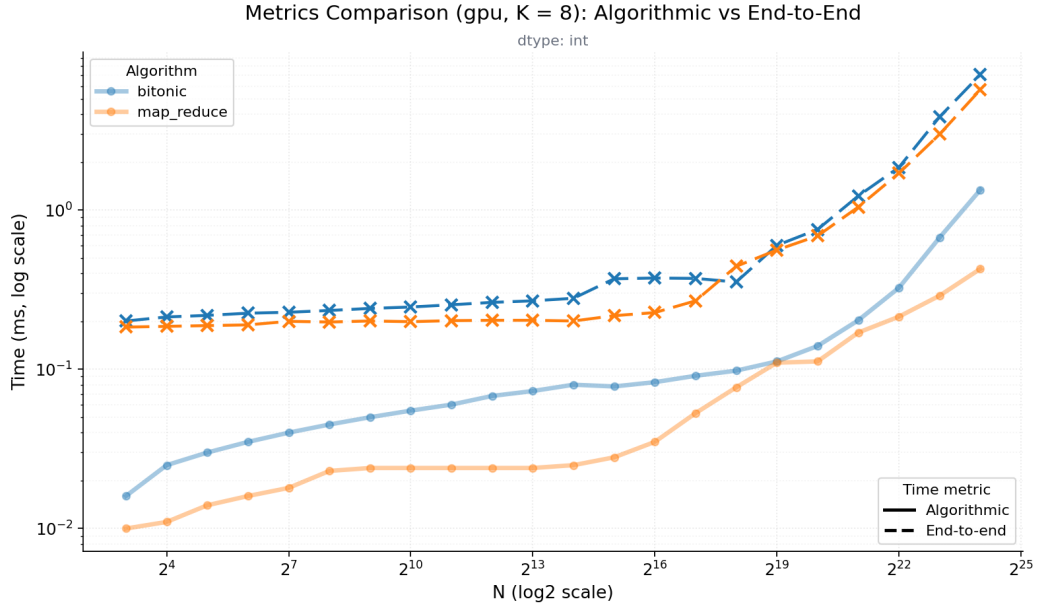


Figure 7.9: Algorithmic and end-to-end time on the graphics unit, for k equal to 8. The distance between the two curves of each algorithm is the cost of transfers and kernel launches.

Before the energy measurements, figure 7.9 shows why the two scopes diverge so much on the graphics unit. The end-to-end curve stays steady near three tenths of a millisecond up to about 2^{16} elements, where the kernel is essentially invisible and the time is spent on transfers and launches. Even at the largest input, the end-to-end time remains a multiple of the algorithmic time, about five times for bitonic sort and thirteen for map-reduce.

The energy picture differs substantially from that of the times, because of the symmetric measurement scope (section 3.3.1). Each architecture is charged the total cost of solving the problem.

At the largest input and small k , map-reduce on the processor consumes about 19 mJ per run, the most economical implementation. The corresponding ones on the graphics unit consume from 108 mJ to 128 mJ, up to seven times more, despite being faster in the algorithmic scope. The most expensive implementation of all is bitonic sort on the graphics unit. The corresponding one on the processor costs 118 mJ.

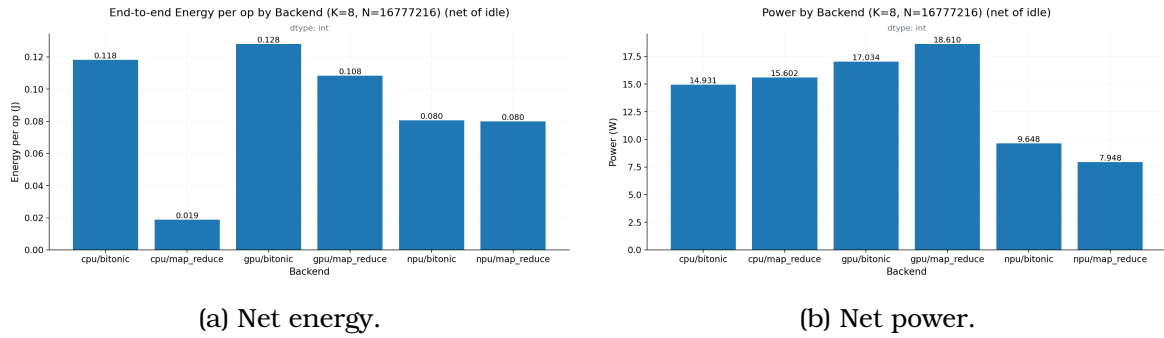


Figure 7.10: Net energy and power by architecture and algorithm, for k equal to 8.

Taken together, figures 7.10a and 7.10b bring out a distinction that is easily misread, especially for the neural processing unit. It has the lowest power draw of all implementations, with its map-reduce at 7.9 W against 15.6 W for the corresponding one on the processor and 18.6 W for the graphics unit. It consumes, however, more than four times the energy of map-reduce on the processor, 0.080 J against 0.019 J. Energy is the product of power and time, and the unit needs about eight times the time for the same work. The forty-nine-percent advantage in power is thus wiped out by the disadvantage of the eightfold duration. The principle that moving data costs more energy than the computation itself [10] holds here too, but the reuse that would amortise it is missing. A low-power architecture pays off energetically only if it finishes in comparable time, and that does not happen here.

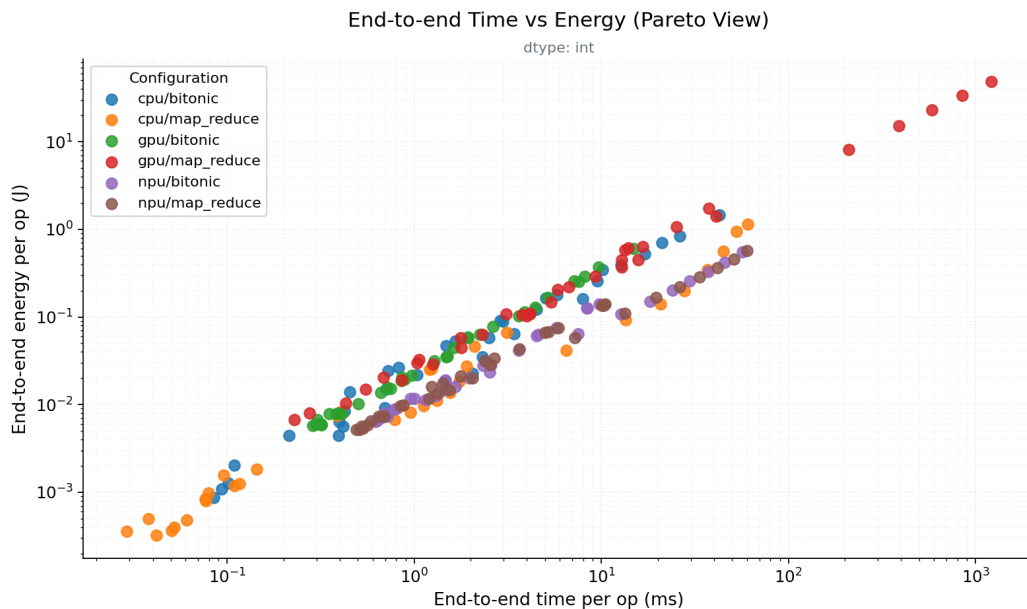


Figure 7.11: End-to-end time and total energy of each measured case. Both quantities are minimised, so the preferable cases lie toward the bottom-left corner.

Figure 7.11 places each case on a plane of two criteria minimised simultaneously. A case is dominated by another when the latter is no worse in either of the two and better in at least one, so there is no reason ever to choose it. The cases dominated

by no other form the Pareto front. There, improving one criterion requires giving ground on the other.

The result is illuminating precisely because it is negative. The measurements form a narrow diagonal band, with a correlation coefficient of 0.97 between the logarithms of time and energy. Of the 209 cases only two belong to the front, both the same implementation, map-reduce on the processor at two neighbouring sizes. There is no substantial trade-off between speed and energy. The fastest configuration is also the most economical.

The reason follows from the two preceding figures. Power varies between implementations by barely a factor of 2.3, while time varies by four orders of magnitude. When one factor of a product is nearly constant, the product follows the other. Trading longer duration for lower power, a genuine design lever in other systems, simply does not exist here.

All cases on the front run on the processor. No configuration of the graphics unit or the neural processing unit is Pareto-optimal anywhere in the parameter space.

7.6 Effect of data type

Implementation	half	int	uint	float	double
CPU bitonic	4.60	8.01	7.93	8.01	14.03
CPU map-reduce	0.39	1.23	1.22	1.22	2.35
CPU reference	13.27	4.65	4.63	5.50	5.54
GPU bitonic	0.88	1.34	1.37	1.34	2.89
NPU bitonic	7.71	8.17	8.37	8.23	25.38

Table 7.2: Algorithmic time by data type, in ms, for 2^{24} elements and k equal to 8.

The implementations of this work track the width of the type, as expected of memory-bound algorithms. Half precision is consistently the fastest; bitonic sort on the processor completes in a little over half the time of single precision, precisely because it moves half the bytes at the same rate of comparisons.

The reference implementation behaves the opposite way, with half precision recording the lowest performance. The difference reveals that the library is limited primarily by the cost of the scalar comparison (which for half precision includes a conversion), rather than by memory. The ranking among types is therefore a characteristic of the implementation, independent of the inherent limitations of the hardware.

Double precision on the neural processing unit stands out, at three times the time of any other type. The overhead is due entirely to the additional correction pass of section 6.4.3, necessary because the 32-bit key is not enough for 64-bit values.

The combination of the two opposite behaviours produces the only point in the whole work where the neural processing unit outperforms its library. For half precision, bitonic sort on the unit is faster than the library in twenty-nine of the measured cases, with the largest gap reaching a factor of two and a half. The phenomenon

appears more weakly in the other two floating-point types, and not at all in the integer ones.

The recorded advantage stems more from the library's weakness in handling half precision than from any inherent superiority of the unit. The array's kernel works on 32-bit integers regardless of type, shifting the conversion cost to the vectorised encoding stage on the host (section 6.4.2). The library, in contrast, performs scalar comparisons with an intermediate conversion. Offloading pays off here because the established solution falls short with this data type, but this does not mean the unit is stronger in general.

7.7 Sensitivity to the input distribution

Through normalisation against the uniform distribution, the nine combinations can be compared on the same plot, even though their absolute execution times differ by orders of magnitude. In the figure below, an implementation unaffected by the data appears as five equal bars of unit height. A sensitive one diverges in both directions.

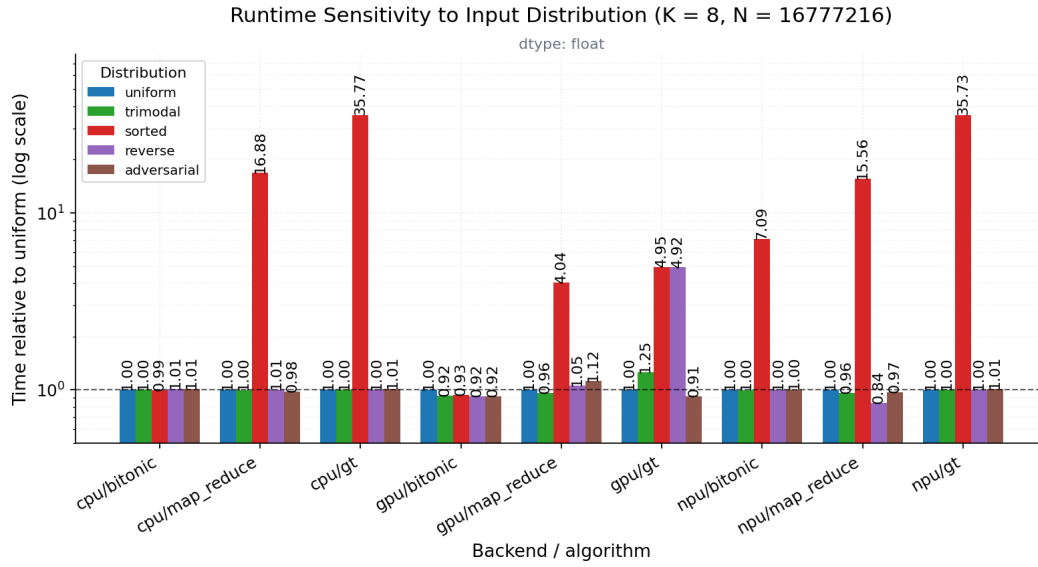
Bitonic sort, on both the processor and the graphics unit, stays practically unaffected by the data distribution for both values of k , with a maximum variation of about 9%. This stability makes it an ideal control indicator. Any substantial change in its execution time would be due solely to factors external to the measurement and not to the inherent behaviour of the algorithm. Bitonic sort on the neural processing unit also stands out here. Unlike the other two architectures it is sensitive to the distribution, with a variation factor of fifty-three. The sensitivity does not come from the array, which executes a fixed network, but from the heap-based merge the host performs, which dominates the time (section 6.4.1).

At the opposite end, the heap-based implementations show considerable sensitivity to the distribution, regardless of who wrote the code. The reference library varies by a factor of seventy-two between ascending and descending input. Our own map-reduce on the processor, by a factor of twenty-five, the same mechanism as in section 4.3. Sensitivity to the data is a property of the algorithm and is unavoidable for any selection method based on a heap.

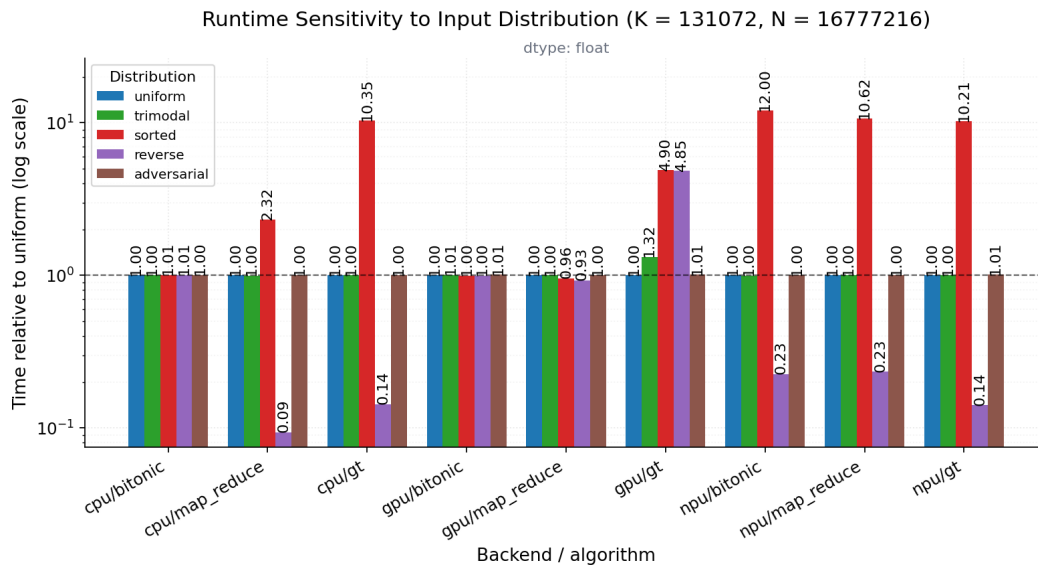
The graphics unit's reference library shows a third, distinct pattern. It slows down by nearly the same factor in both directions, 4.95 for ascending and 4.92 for descending, without favouring one as the heaps do. Sorting does not change the distribution of the values (section 3.7.2), so this is not the radix-adversarial digit distribution (section 2.2.4). Instead it is a matter of position. The candidates that survive the critical bucket end up clustered at one end of the array or the other, which symmetrically burdens the library's parallel compaction step regardless of which end.

Juxtaposing the two values of k isolates the cause of the sensitivity of map-reduce on the graphics unit. While the implementation is highly sensitive for $k=8$ (sorted input slows it down by a factor of 4.0), it stays nearly unaffected for $k=131072$ (the same input changes it by only 4%). The same code, with the same data, changes behaviour because the limiting factor changes there. At large k , performance is now determined by the lack of parallelism (section 5.3.1) and not by the rejection rate.

Naturally, the data distribution stops mattering.



(a) k=8.



(b) k=131072.

Figure 7.12: Sensitivity of time to the input distribution, for single precision and 2^{24} elements. Each bar gives the time of one distribution relative to the time of the uniform one for the same implementation. One means complete independence from the data.

Chapter 8

Conclusions and future work

8.1 Conclusions

This work implemented two top-k selection algorithms on three architectures, using a common harness for their evaluation. This section summarises the conclusions of chapter 7, focusing on the findings that hold regardless of the underlying hardware.

Data movement. For small k , every combination of algorithm and architecture is limited by bandwidth and not by the computational capability of the devices (section 7.4). Performance depends solely on reducing the volume of data moved. Bitonic sort on the graphics unit confirms this clearly. Although it uses a larger fraction of the bandwidth ceiling than the reference library, it ultimately falls behind because it is forced to move several times as much data.

The ratio n/k . The fastest implementation for $k=8$ becomes the slowest for $k=131072$ regardless of platform (section 7.2.2). The performance of the sorting network stays far more stable. The heap algorithm starts filtering elements only once it is full, which caps the available parallelism by the ratio n/k . Similar behaviour has also been observed in the literature [15], highlighting the need for adaptive algorithm switching depending on the input data.

The measurement boundary. While the graphics unit wins in pure kernel execution time at the extreme sizes of the experiment, the overall end-to-end measurement shows the processor to be faster in every case. Shifting the measurement boundary completely changes the final ranking. The evaluation of an implementation must account for the cost of data transfer; otherwise the comparison of whether offloading to the card pays off remains incomplete.

Power and energy. The neural processing unit has the lowest power draw of all implementations and at the same time the highest consumption, since duration dominates the product (section 7.5).

For top-k selection with small values of k , offloading the computation to an accelerator proves disadvantageous in both energy and time compared to running

map-reduce on the processor. Architectures such as NPUs (section 2.3.3) presuppose high local data reuse to amortise their transfer cost. Top-k selection does not offer this, so most of the time is spent preparing the data, leaving the device idle (section 6.4.1).

The only exception is bitonic sort on the neural processing unit for half-precision numbers, where it surpasses the corresponding reference implementation. The advantage is attributed to the incomplete optimisation of the processor’s library for this particular data type (section 7.6).

8.2 Limitations

The comparison with `std::partial_sort` on the processor involves an inherent asymmetry. The library runs strictly single-threaded, unlike the implementations of this work, which exploit parallelism (section 4.5). The recorded speed-up results jointly from both the algorithmic improvement and the use of multiple threads. This choice stems from the intention to compare the new implementation with the tools a programmer has immediately available, but the difference should be kept in mind when reading the results.

In addition, the volume of data counted for the reference libraries is merely a lower bound. Because their internal memory passes are not directly observable, the ceiling-utilisation percentages reported in section 7.4.1 are underestimates.

For the neural processing unit no native top-k library is available for direct comparison. The comparison was made against the host, answering the practical question of whether offloading is worthwhile, without evaluating the maximum theoretical performance of the device itself. Correspondingly, the phase analysis of section 6.4.1 comes from separate runs with debug enabled rather than from the standard sweep, so its percentages rest on a smaller sample of repetitions than the other measurements of appendix B.

Finally, the experimental space was limited to a specific system and specific ranges of values. Although the qualitative trends around the ratio n/k and the role of the measurement boundary are expected to generalise to similar systems, the exact numerical factors remain closely tied to this particular test platform.

8.3 Future work

In terms of system-level optimisations, executing multiple simultaneous top-k queries is the main future extension, especially for the neural processing unit. Since the main obstacle lies in preparing and dispatching the data, batching the calls would help decisively in amortising this cost. Such repeated queries dominate the applications of section 2.1. In the same direction, exploiting mechanisms for direct device access to system memory, avoiding explicit copying of data by the host, could significantly improve overall latency. Implementing such a capability depends to a large extent on the specifications and interfaces of each platform.

As for algorithmic extensions, a mechanism for dynamic algorithm switching would directly exploit the finding about the ratio n/k . Knowing now how the ratio affects

performance, an adaptive implementation could automatically choose between a heap and a sorting network to ensure smooth performance across the whole range of sizes. Furthermore, for larger values of k , where selection approaches full sorting, handling inputs that do not fit in device memory is a natural continuation, turning stream management from a responsibility of the harness into an organic part of the algorithm itself.

Finally, the evaluation harness developed in this work can be used in the future to study other workloads as well. Separating algorithmic time from transfer time, recording energy uniformly across the different architectures and analysing through the roofline model offer a reusable framework, capable of being applied to any other problem considered for offloading to heterogeneous hardware.

Appendix A

Acronyms

AIE	AI Engine
API	Application Programming Interface
AVX	Advanced Vector Extensions
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
D2H	Device to Host
DRAM	Dynamic Random Access Memory
FIFO	First In First Out
GPU	Graphics Processing Unit
H2D	Host to Device
MLIR	Multi-Level Intermediate Representation
NPU	Neural Processing Unit
NVML	NVIDIA Management Library
RAPL	Running Average Power Limit
SIMD	Single Instruction Multiple Data
SIMT	Single Instruction Multiple Threads
SM	Streaming Multiprocessor
TOPS	Tera Operations Per Second
XRT	Xilinx Runtime

Appendix B

Reproducing the measurements

This appendix describes the steps for reproducing the measurements of this work. The procedure covers installing the tools, running the code, and generating the plots and the report. The commands concern the system of section 3.1 and are run from the root of the repository (<https://github.com/ioannisam/topk>).

B.1 Software requirements

All three implementations require a C++ compiler that supports the C++20 standard, as well as cmake version 3.18 or newer. Vector instructions are not a compilation requirement. The choice between AVX-512, AVX2 and scalar execution is made at run time, based on the capabilities reported by the processor. Because the measurements were made on a machine with AVX-512, its absence leads to a different code path and the results are not directly comparable.

The graphics unit requires the CUDA toolkit, which includes the nvcc compiler and the CUB library. Power measurements additionally require the driver's NVML library.

The neural processing unit requires the XRT runtime stack and the XDNA plugin. Compiling the kernels needs MLIR-AIE in a Python environment. This environment must reside at the root of the repository under the name `npu_venv`, since that is where the run scripts look for it. The user must belong to the device's access groups and have a sufficient locked-memory limit; otherwise the device is not recognised. The `XILINX_XRT` variable must point to the XRT installation.

Energy measurement reads the RAPL counters from `/sys/class/powercap`, which requires administrator privileges. In a single run without them, the measurement is explicitly declared unavailable instead of recording zeros (section 3.3.3). The automated sweep, in contrast, is aborted entirely to avoid producing an empty dataset. Finally, the analysis infrastructure requires the repository's Python libraries. The corresponding environment is created automatically on the first run.

B.2 Compilation

All implementations are compiled with a single command:

```
make build-all
```

Each architecture can also be compiled on its own, with the targets `build-cpu`, `build-gpu` and `build-npu`, which is useful on a system that does not have all three units. The resulting executables are placed in the `build` directory, two per architecture: one for the top-k measurements and one for the ceiling measurements of section 3.4.

Compilation for the neural processing unit additionally produces the hardware configuration files. Because their absence leads to the corresponding measurements being silently skipped, their presence is checked explicitly:

```
make verify-artifacts
```

The check examines the two configuration files and the three top-k executables. It presupposes a full compilation, so it fails automatically if only one architecture has been compiled.

B.3 Running the experiments

The recommended way to reproduce the results is to run the whole procedure with a single command:

```
make benchmark GPU_W=50
```

This command ensures the correct sequence of steps. If the conditions are pinned after the fact or old executables are used, the results lose their comparability. The command enforces the proper order and prevents omissions.

The sequence comprises six stages. First, the old results and executables are deleted. Compilation and the check of the files follow (section B.2). In the third stage the system conditions are pinned and access to the energy counters is granted. The four measurement sweeps follow, then the generation of the plots and, finally, the restoration of the system.

The procedure first checks the availability of `sudo` and of the `npu_venv` environment. If either is missing, the command terminates immediately, without altering the system or the data.

Administrator privileges are requested once and remain active. The conditions are always restored at the end, even in case of an error or an interruption by the user, preventing the system from being left in a pinned state. The failure of an individual sweep does not stop the rest; it is simply logged as a warning.

The `GPU_W` argument sets the power limit of the graphics unit, which must remain constant (section 3.8). The procedure deletes the existing results immediately, so the user must first keep copies of any data they want to preserve.

B.3.1 Running individual steps

For a partial or repeated run, each stage is also available separately. The system conditions are controlled with:

```
make pin ARGS=50      # governor, boost off, GPU cap 50 W
make pin-show         # report the current state
make unpin            # restore governor, boost, GPU cap
```

The four independent sweeps write their raw results to separate files:

```
make measure-cases    # timing over the parameter space
make measure-energy    # energy, requires RAPL access
make measure-dists    # input-distribution sensitivity
make measure-roofline # bandwidth, compare-exchange roofs
```

The targets accept additional arguments through the ARGS variable, restricting the parameter space. This makes it easy to check the installation quickly. In a manual run, the user takes responsibility for the correct order and for keeping the conditions constant.

B.3.2 Execution parameters

Table B.1 summarises the parameters of each executable, the permitted values, the defaults and the values used in the sweeps.

The defaults often differ from the final sweep values. The value range was set up to 10^9 instead of the default, to avoid an excessive accumulation of ties in inputs of millions of elements. Many ties would artificially favour the algorithms that reject elements quickly.

This range applies to four of the five types. For half, the executable clamps the bounds to ± 65504 , the largest finite value of the type. In this case ties remain unavoidable, because of the narrow range and the inability of half to represent all integers above 2048. This property stems from the type itself and directly affects the interpretation of the results.

The random seed is generated deterministically from the q and k of each case. This ensures reproducibility without the same input ever being given under different conditions.

The main sweep uses the values of the fourth column. The special sweeps restrict the parameters: the distribution sweep examines q from 20 to 24, the float and half types, and the k extremes 8 and 131072, with two seeds per case. The energy sweep examines q from 16 to 24 with three repetitions, skipping the correctness check so as not to inflate the consumed power needlessly.

Parameter	Values	Default	In the sweeps	Meaning
q	integer 0-63	—	3-24	Input size $n = 2^q$ (the only mandatory one)
k	positive integer	n	8, 256, 4096, 131072	Number of requested elements (capped at n)
mode	min max	max	max	Selects the smallest or largest elements
dtype	int uint float double half	int	all five	Element type
algo	bitonic map_reduce gt	bitonic	all three	Algorithm to run
run	full trunc both	trunc	trunc	Variant of the bitonic network
seed	non-negative integer	42	generated from q, k	Seed of the random-number generator
dist	uniform normal trimodal sorted reverse adversarial	uniform	uniform (all five in the distribution sweep)	Input distribution
min / max	integers	0 / 1000	0 / 10^9	Value range (requires $\max \geq \min$, clamped to the range of the type)
threads	positive integer	0 (automatic)	8	Number of threads (ignored by the NPU)
verify	true false	false	true	Check against the verification implementation
debug	true false	true	false	Verbose diagnostic output

Table B.1: The execution parameters and their defaults.

B.4 Generating the plots

The analysis infrastructure reads the raw results, converts them into CSV tables and generates the plots:

```
make plot ARGS="--plot all"
```

The `-plot` argument selects the plots and `-input` the sweep (the default is the timing sweep). Because the other plots use their own files, the combined targets are recommended, which run the measurement and the generation directly:

```
make benchmark-cases
make benchmark-energy
make benchmark-dists
make benchmark-roofline
```

The files are organised by data type and category. Plots that concern only the hardware (e.g. the bandwidth scale) are stored in a separate directory, since they do not depend on the type.

Two parameters determine the presentation. `-energy-metric` selects between gross and net energy. The sweeps of this work use the net figure (section 3.3.3). The `-error-bars` parameter sets the dispersion band (standard deviation, deciles or none). The band is drawn only if there are at least two samples per point, that is, in the distribution sweep (two seeds) and the energy sweep (three repetitions). In the automated run, the bands are disabled entirely.

Bibliography

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [2] N. Sismanis, N. Pitsianis, and X. Sun, “Parallel search of k-nearest neighbors with synchronous operations,” in *2012 IEEE Conference on High Performance Extreme Computing (HPEC)*, Waltham, MA, USA, 2012, pp. 1–6.
- [3] J. Zhang, A. Naruse, X. Li, and Y. Wang, “Parallel top-K algorithms on GPU: A comprehensive study and new methods,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC ’23)*, Denver, CO, USA, 2023.
- [4] K. E. Batcher, “Sorting networks and their applications,” in *Proceedings of the AFIPS Spring Joint Computer Conference*, 1968, pp. 307–314.
- [5] A. Shanbhag, H. Pirk, and S. Madden, “Efficient top-K query processing on massively parallel hardware,” in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, Houston, TX, USA, 2018, pp. 1557–1570.
- [6] D. A. Palmer and M. Florea, “Neural processing unit,” U.S. Patent US8655815B2, assigned to the University of California, San Diego, 2014, application US13/011,727, filed 21 January 2011; granted 18 February 2014.
- [7] H. Esmailzadeh, A. Sampson, L. Ceze, and D. Burger, “Neural acceleration for general-purpose approximate programs,” in *45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2012, pp. 449–460.
- [8] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, “DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning,” in *19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014, pp. 269–284.
- [9] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *44th International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12.
- [10] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks,” in *43rd International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.
- [11] AMD, “Versal adaptive SoC: The industry’s first adaptive compute acceleration platform,” AMD, White Paper, 2021, <https://www.amd.com/>

content/dam/amd/en/documents/products/adaptive-socs-and-fpgas/versal/adaptive-soc-paper.pdf.

- [12] A. Rico, S. Pareek, J. Cabezas, J. Noguera *et al.*, “AMD XDNA NPU in Ryzen AI processors,” *IEEE Micro*, vol. 44, no. 6, 2024.
- [13] AMD/Xilinx, “MLIR-AIE: An MLIR-based toolchain for AMD AI engine-enabled devices,” <https://github.com/Xilinx/mlir-aie>, 2024.
- [14] NVIDIA, “CCCL: CUDA core compute libraries,” <https://github.com/NVIDIA/cccl>, 2024.
- [15] C. Lin, M. Yang, C. Wang, and C. Huang, “Adaptive top-K selection: Eliminating performance cliffs across all K values on AMD GPUs,” AMD ROCm Blogs, Feb. 2026, <https://rocm.blogs.amd.com/software-tools-optimization/adaptive-topk/README.html>.
- [16] Y. Samaga, V. Yerram, S. R. Babbula, P. Jain, and P. Netrapalli, “A faster generalized two-stage approximate top-K,” *Transactions on Machine Learning Research*, 2026, <https://openreview.net/forum?id=izqZ1Crpjz>.
- [17] AMD, “Bitonic SIMD sorting on AI engine for float datatypes,” Vitis Tutorials: AI Engine Development, 2024.1, 2024, <https://docs.amd.com/r/2024.1-English/Vitis-Tutorials-AI-Engine-Development/Bitonic-SIMD-Sorting-on-AI-Engine-for-float-Datatypes>.
- [18] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [19] Intel, “Intel intrinsics guide,” <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>, 2024.
- [20] AMD, “IRON: A close-to-metal programming framework for AMD NPUs,” <https://github.com/amd/IRON>, 2024.
- [21] —, “AI engine kernel and graph programming guide (UG1079): Casting and datatype conversion,” 2024, <https://docs.amd.com/r/en-US/ug1079-ai-engine-kernel-coding/>.